Network Analysis of Internal Migration in Honduras

Applying Network Concepts and Algorithms to Remittance-Based Migration Flows

Elvis Casco

2026-03-01

Table of contents

1 Motivation **2**

2 Setup **2**

3 1 — Constructing the Migration Network **3**

3.1 1.1 — Department-Level Network (Aggregated) . . . 3

3.2 1.2 — Municipality-Level Network . . . 6

3.3 1.3 — Visualising the Department Network . . . 7

4 2 — Degree Distribution and Basic Network Statistics (Unit 1) **9**

4.1 2.1 — Department Network Statistics . . . 9

4.2 2.2 — Municipality Network Degree Distribution . . . 12

5 3 — Centrality Analysis (Unit 1) **14**

5.1 3.1 — Department-Level Centrality . . . 14

5.2 3.2 — Municipality-Level Centrality (Top 20) . . . 17

6 4 — Comparison to Random Graph Models (Unit 2) **20**

6.1 4.1 — Erdős–Rényi Benchmark . . . 20

6.2 4.2 — Configuration Model Benchmark . . . 22

7 5 — Community Detection (Unit 3) **24**

7.1 5.1 — Community Detection on the Department Network . . . 24

8 6 — Migration as a Spreading Process (Unit 5) **29**

8.1 6.1 — SI Model: Migration Contagion on the Department Network . . . 29

8.2 6.2 — Threshold Model: Complex Contagion (Unit 5.2) . . . 33

9 7 — Optimal Seeding for Development Interventions (Unit 5.2) **36**

10 8 — Opinion Dynamics on the Migration Network (Unit 6) **39**
10.1 8.1 — DeGroot Model . . . 39
10.2 8.2 — Influence Vector and Spectral Analysis . . . 43
10.3 8.3 — Friedkin-Johnsen Model . . . 45

11 9 — Routing Analogy: Migration as a Flow Network (Unit 4) **47**
11.1 9.1 — Migration Flow Game Setup . . . 47

12 10 — Summary of Network Analysis Findings **53**

13 Conclusion **56**
13.1 Development Policy Recommendations . . . 57

14 References **58**

1 Motivation

The dataset `remesas_year_munic.csv` records remittance flows between origin (birth) and destination (receipt) municipalities in Honduras (2020–2024). When a person was born in municipality A but receives remittances in municipality $B \neq A$, this strongly suggests **internal migration** from A to B.

This creates a natural **directed, weighted graph** where:

- **Nodes** = municipalities (or departments)
- **Directed edges** = migration flows from birth municipality to receipt municipality
- **Edge weights** = number of migrant beneficiaries (N) or total USD

We apply concepts from the Networks I course (Pau Milán, BSE 2026) to analyse this migration network:

Course Unit	Application to Migration Data
Unit 1 — Graph fundamentals	Degree distributions, centrality, clustering
Unit 2 — Random graph models	Compare migration network to ER/configuration null models
Unit 3 — Community detection	Discover migration communities (clusters of municipalities that exchange people)
Unit 5 — Diffusion & cascades	Model migration as a spreading process; threshold models
Unit 5.2 — Optimal seeding	Identify optimal municipalities for development interventions
Unit 6 — Opinion dynamics	DeGroot / Friedkin-Johnsen learning on the migration network

---

2 Setup

```python
import numpy as np
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from matplotlib.colors import Normalize
from scipy.sparse.linalg import eigs
import scipy.sparse as sp
from collections import Counter
import warnings
warnings.filterwarnings('ignore')

#   Load remittance data
df = pd.read_csv("remesas_year_munic.csv")

#   Department names
dept_names = {
    1: "Atlántida", 2: "Colón", 3: "Comayagua", 4: "Copán",
    5: "Cortés", 6: "Choluteca", 7: "El Paraíso",
    8: "Francisco Morazán", 9: "Gracias a Dios", 10: "Intibucá",
    11: "Islas de la Bahía", 12: "La Paz", 13: "Lempira",
    14: "Ocotepeque", 15: "Olancho", 16: "Santa Bárbara",
    17: "Valle", 18: "Yoro"
}

#   Municipality-level data
mun_data = pd.read_csv("data/municipality_data.csv")
mun_data['mun_id'] = mun_data['codDepto'].astype(str).str.zfill(2) + \
                     mun_data['codMun'].astype(str).str.zfill(2)

print(f"Remittance records: {len(df):,}")
print(f"Years covered: {sorted(df['Year'].unique())}")
print(f"Municipalities in reference data: {len(mun_data)}")
```

```
Remittance records: 94,544
Years covered: [np.int64(2020), np.int64(2021), np.int64(2022), np.int64(2023), np.int64(2024)]
Municipalities in reference data: 298
```

---

3 1 — Constructing the Migration Network

3.1 1.1 — Department-Level Network (Aggregated)

We first build a **department-level** directed graph. Each edge $(i \to j)$ represents people born in department i who receive remittances in department j, weighted by total beneficiaries N. We exclude self-loops (same department) to focus on inter-department migration.

```python
#   Aggregate flows at the department level
dept_flows = df.groupby(['codDeptoNac', 'codDepto']).agg(
```

```
    N=('N', 'sum'),
    USD=('USD', 'sum')
).reset_index()

# Exclude self-loops (same department)
dept_migration = dept_flows[dept_flows['codDeptoNac'] != dept_flows['codDepto']].copy()
dept_migration['dept_born'] = dept_migration['codDeptoNac'].map(dept_names)
dept_migration['dept_recv'] = dept_migration['codDepto'].map(dept_names)

#   Build directed graph
G_dept = nx.DiGraph()

# Add all 18 departments as nodes
for code, name in dept_names.items():
    G_dept.add_node(name, code=code)

# Add weighted edges
for _, row in dept_migration.iterrows():
    G_dept.add_edge(row['dept_born'], row['dept_recv'],
                    weight=row['N'], usd=row['USD'])

print(f"Department network: {G_dept.number_of_nodes()} nodes, "
      f"{G_dept.number_of_edges()} edges")
print(f"Density: {nx.density(G_dept):.4f}")
print(f"Total inter-department migrants: {dept_migration['N'].sum():,.0f}")
```

```
Department network: 18 nodes, 306 edges
Density: 1.0000
Total inter-department migrants: 12,897,536
```

The department network has 18 nodes (one per department) and a **density** that tells us how many of all possible migration corridors are actually used. A high density (close to 1.0) would mean every department sends migrants to every other; a low value would indicate that migration is concentrated in a few corridors. Honduras, being a small country with two dominant urban poles (Tegucigalpa and San Pedro Sula), likely has moderate density — most departments connect to the two poles but not to each other.

```
#   Heatmap of department-to-department flows
depts_ordered = sorted(dept_names.values())
n_d = len(depts_ordered)
flow_matrix = np.zeros((n_d, n_d))
d_idx = {d: i for i, d in enumerate(depts_ordered)}

for _, row in dept_migration.iterrows():
    i = d_idx.get(row['dept_born'])
    j = d_idx.get(row['dept_recv'])
    if i is not None and j is not None:
        flow_matrix[i, j] = row['N']

fig, ax = plt.subplots(figsize=(10, 8))
im = ax.imshow(np.log10(flow_matrix + 1), cmap='YlOrRd', aspect='auto')
ax.set_xticks(range(n_d))
ax.set_yticks(range(n_d))
ax.set_xticklabels(depts_ordered, rotation=90, fontsize=7)
```

```
ax.set_yticklabels(depts_ordered, fontsize=7)
ax.set_xlabel('Destination (receipt) department')
ax.set_ylabel('Origin (birth) department')
ax.set_title('Inter-Department Migration Flows (log  scale)\n'
             'Bright rows = departments sending many migrants;\n'
             'Bright columns = departments receiving many migrants')
cbar = plt.colorbar(im, ax=ax, label='log (N + 1)')
plt.tight_layout()
plt.show()
```

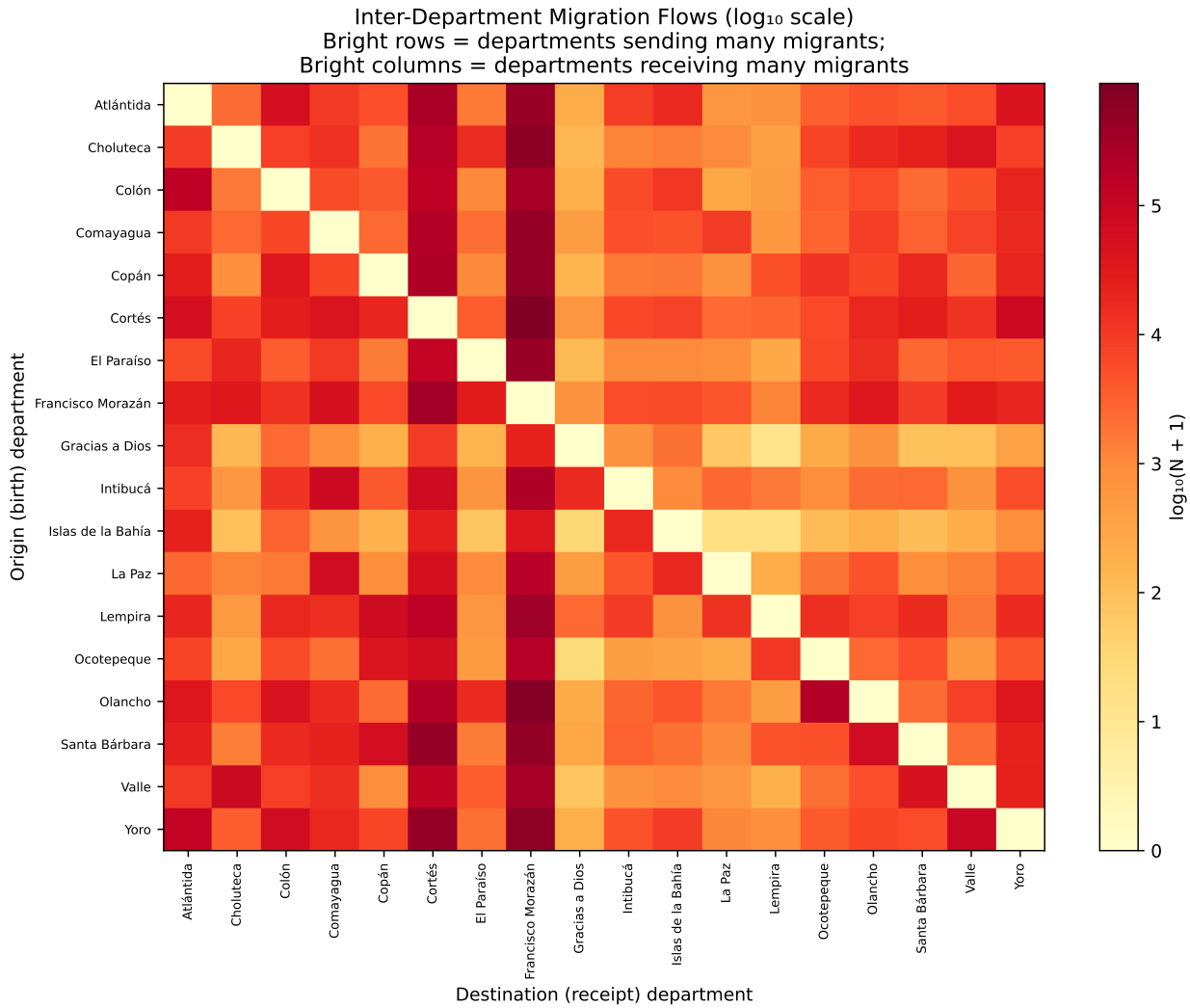

Figure 1: Origin–Destination heatmap of inter-department migration

Reading the Heatmap

Each cell (i, j) in the heatmap represents the number of people born in department i who receive remittances in department j. **Bright rows** indicate departments with high out-migration across many destinations. **Bright columns** indicate popular receiving departments — we expect Francisco

Morazán (Tegucigalpa) and Cortés (San Pedro Sula) to have the brightest columns, confirming their role as the nation's primary migration attractors.
The heatmap reveals the **asymmetry** of migration: flows are predominantly from rural/peripheral departments toward the two urban centres, not bidirectional. This has direct implications for development policy: resources flowing into Tegucigalpa and San Pedro Sula create pressure on urban infrastructure, while origin departments lose human capital.

3.2 1.2 — Municipality-Level Network

We also build a finer-grained **municipality-level** network where nodes are individual municipalities. This network is much larger and sparser.

```
#   Create municipality IDs
df['origin_id'] = df['codDeptoNac'].astype(str).str.zfill(2) + \
                  df['codMunNac'].astype(str).str.zfill(2)
df['dest_id'] = df['codDepto'].astype(str).str.zfill(2) + \
                df['codMun'].astype(str).str.zfill(2)

# Aggregate across years, exclude self-loops
mun_flows = df[df['origin_id'] != df['dest_id']].groupby(
    ['origin_id', 'dest_id']
).agg(N=('N', 'sum'), USD=('USD', 'sum')).reset_index()

#   Build directed graph
G_mun = nx.DiGraph()

# Add nodes with attributes
for _, row in mun_data.iterrows():
    mid = row['mun_id']
    G_mun.add_node(mid,
                   dept=row['ADM1_ES'],
                   name=row['ADM2_ES'],
                   pop=row['T_TL'],
                   lon=row['lon'],
                   lat=row['lat'])

# Add edges
for _, row in mun_flows.iterrows():
    if row['origin_id'] in G_mun and row['dest_id'] in G_mun:
        G_mun.add_edge(row['origin_id'], row['dest_id'],
                       weight=row['N'], usd=row['USD'])

# Remove isolated nodes (no inter-municipal migration)
isolates = list(nx.isolates(G_mun))
G_mun_connected = G_mun.copy()
G_mun_connected.remove_nodes_from(isolates)

print(f"Municipality network: {G_mun.number_of_nodes()} nodes, "
      f"{G_mun.number_of_edges()} edges")
print(f"After removing isolates: {G_mun_connected.number_of_nodes()} nodes, "
      f"{G_mun_connected.number_of_edges()} edges")
print(f"Network density: {nx.density(G_mun_connected):.6f}")
```

```
Municipality network: 298 nodes, 26102 edges
After removing isolates: 298 nodes, 26102 edges
Network density: 0.294918
```

3.3 1.3 — Visualising the Department Network

```
#   Use geographic coordinates for layout
# Group by codDepto (integer) to avoid accent-mismatch with ADM1_ES
dept_coords = mun_data.groupby('codDepto').agg(
    lon=('lon', 'mean'), lat=('lat', 'mean')
).to_dict('index')

pos = {}
for code, name in dept_names.items():
    if code in dept_coords:
        pos[name] = (dept_coords[code]['lon'], dept_coords[code]['lat'])

#   Node sizes proportional to in-degree (weighted)
in_strength = dict(G_dept.in_degree(weight='weight'))
max_in = max(in_strength.values()) if in_strength else 1
node_sizes = [300 + 3000 * in_strength.get(n, 0) / max_in for n in G_dept.nodes()]

#   Edge widths proportional to flow
edge_weights = [G_dept[u][v]['weight'] for u, v in G_dept.edges()]
max_w = max(edge_weights) if edge_weights else 1
edge_widths = [0.3 + 4 * w / max_w for w in edge_weights]
edge_alphas = [0.2 + 0.6 * w / max_w for w in edge_weights]

fig, ax = plt.subplots(figsize=(12, 9))

# Draw edges with varying alpha
for (u, v), width, alpha in zip(G_dept.edges(), edge_widths, edge_alphas):
    nx.draw_networkx_edges(G_dept, pos, edgelist=[(u, v)],
                           width=width, alpha=alpha,
                           edge_color='steelblue',
                           connectionstyle='arc3,rad=0.1',
                           arrows=True, arrowsize=10, ax=ax)

# Draw nodes
nx.draw_networkx_nodes(G_dept, pos, node_size=node_sizes,
                       node_color=list(in_strength.values()),
                       cmap=plt.cm.YlOrRd, alpha=0.9, ax=ax)
nx.draw_networkx_labels(G_dept, pos, font_size=7, font_weight='bold', ax=ax)

ax.set_title("Honduras Internal Migration Network (Departments)\n"
             "Node size & colour = weighted in-degree (migrant inflow)",
             fontsize=14)
ax.axis('off')
plt.tight_layout()
plt.show()
```

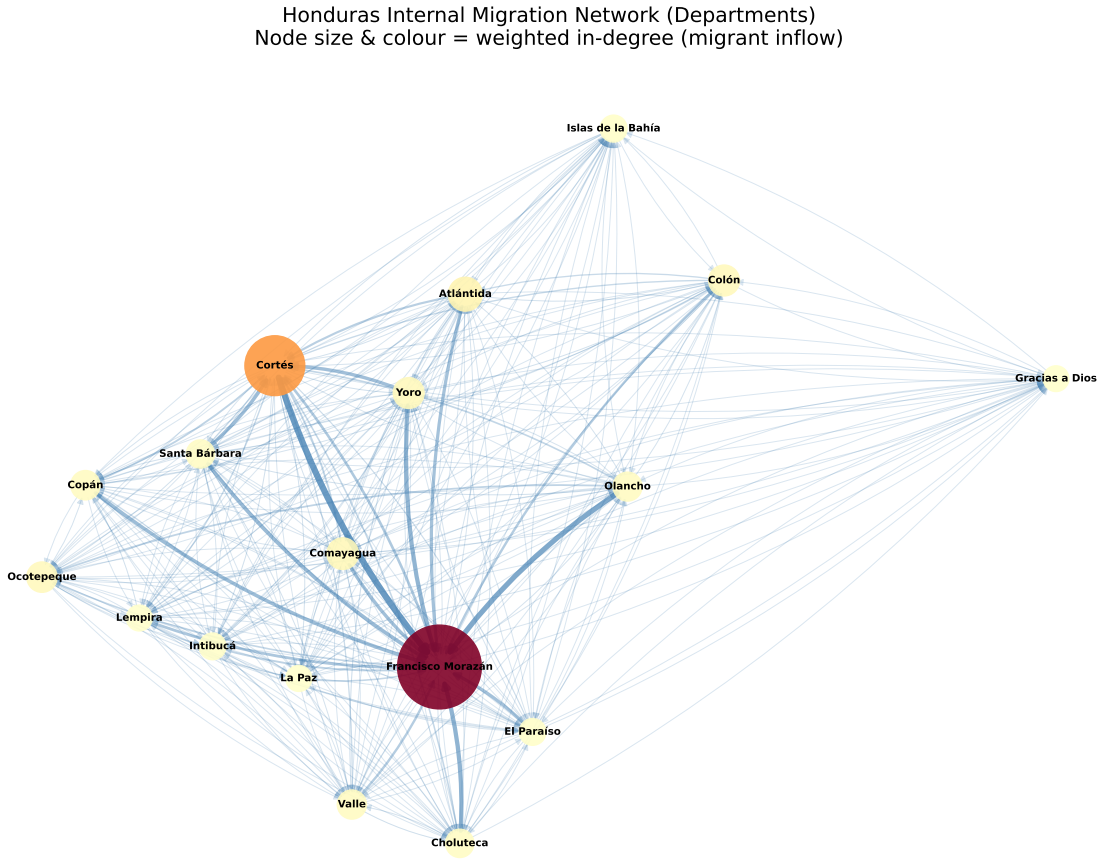

Figure 2: Internal Migration Network of Honduras (Department Level)

Interpreting the Network Map

The geographic network layout reveals several important structural features:

- **Urban gravity:** The two largest nodes (Francisco Morazán and Cortés) sit at the endpoints of the industrial corridor that defines Honduras's economic geography. Their size reflects the overwhelming concentration of migration inflows.
- **Thick corridors:** The widest arrows typically run from western departments (Copán, Santa Bárbara, Lempira) toward Cortés, and from eastern/southern departments (Olancho, El Paraíso, Choluteca) toward Francisco Morazán. These represent the dominant migration pathways.
- **Peripheral isolation:** Departments like Gracias a Dios and Islas de la Bahía appear with small nodes and thin edges, reflecting their geographic isolation and limited participation in internal migration.

Policy implication: The "hub-and-spoke" pattern suggests that Honduras's migration system is heavily **centralised** around two poles. Development policies that invest in **secondary cities** (e.g., Comayagua, La Ceiba, Choluteca) could help redistribute migration pressure and reduce congestion in the two main metropolises. The network perspective highlights that interventions in "bridge" departments (those connecting different migration clusters) may be especially effective.

4 2 — Degree Distribution and Basic Network Statistics (Unit 1)

Following Unit 1 of the course, we compute the fundamental network statistics: degree distribution, density, and average path length. For a directed graph, each node has an **in-degree** (migrants arriving) and an **out-degree** (migrants leaving).

$$k_i^{\text{in}} = \sum_j A_{ji}, \qquad k_i^{\text{out}} = \sum_j A_{ij}$$

4.1 2.1 — Department Network Statistics

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Weighted in-degree (strength)
in_str = dict(G_dept.in_degree(weight='weight'))
out_str = dict(G_dept.out_degree(weight='weight'))

# Unweighted degrees
in_deg = dict(G_dept.in_degree())
out_deg = dict(G_dept.out_degree())

# Plot 1: In-degree vs Out-degree (weighted)
axes[0].scatter(list(out_str.values()), list(in_str.values()),
                s=80, c='steelblue', alpha=0.7)
for dept in G_dept.nodes():
    axes[0].annotate(dept[:4], (out_str[dept], in_str[dept]),
                     fontsize=5, alpha=0.7)
axes[0].set_xlabel('Out-strength (migrants leaving)')
axes[0].set_ylabel('In-strength (migrants arriving)')
axes[0].set_title('Migration Balance by Department')
# Add 45-degree line
maxval = max(max(out_str.values()), max(in_str.values()))
axes[0].plot([0, maxval], [0, maxval], 'r--', alpha=0.5, label='Balance line')
axes[0].legend(fontsize=8)

# Plot 2: In-strength distribution (histogram)
axes[1].hist(list(in_str.values()), bins=15, edgecolor='black', alpha=0.7,
             color='indianred')
axes[1].set_xlabel('Weighted In-degree')
axes[1].set_ylabel('Count')
axes[1].set_title('In-Strength Distribution')

# Plot 3: Out-strength distribution
axes[2].hist(list(out_str.values()), bins=15, edgecolor='black', alpha=0.7,
             color='steelblue')
axes[2].set_xlabel('Weighted Out-degree')
axes[2].set_ylabel('Count')
axes[2].set_title('Out-Strength Distribution')

plt.tight_layout()
plt.show()
```

```
# Summary statistics
print("=== Department Network Summary ===")
print(f"Nodes: {G_dept.number_of_nodes()}")
print(f"Edges: {G_dept.number_of_edges()}")
print(f"Density: {nx.density(G_dept):.4f}")
print(f"Is strongly connected: {nx.is_strongly_connected(G_dept)}")
print(f"Number of strongly connected components: "
      f"{nx.number_strongly_connected_components(G_dept)}")
```

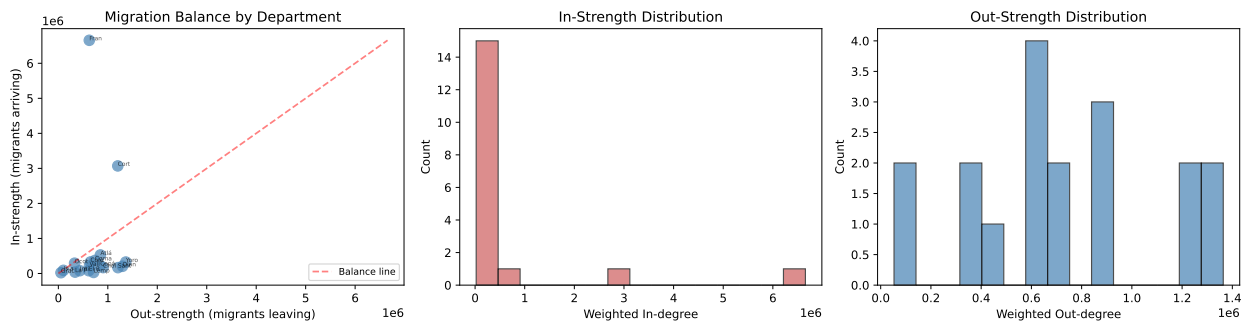

Figure 3: Degree distributions of the department-level migration network

```
=== Department Network Summary ===
Nodes: 18
Edges: 306
Density: 1.0000
Is strongly connected: True
Number of strongly connected components: 1
```

Understanding the Scatter Plot

The **Migration Balance** scatter plot (left panel) is the most revealing: departments **above the 45° line** are net receivers (in-strength > out-strength), while those **below** are net senders. This immediately identifies:

- **Migration magnets:** Cortés (San Pedro Sula) and Francisco Morazán (Tegucigalpa) should appear in the upper-left quadrant — they receive far more migrants than they send.
- **Feeder departments:** Western departments like Copán, Lempira, and Santa Bárbara likely cluster in the lower-right — they send many migrants but receive few.
- **Balanced departments:** Those near the 45° line have roughly symmetric flows, suggesting they serve as both origins and destinations.

```
#   Net migration balance
net_balance = {d: in_str[d] - out_str[d] for d in G_dept.nodes()}
sorted_depts = sorted(net_balance.keys(), key=lambda x: net_balance[x])

colors_bar = ['#e74c3c' if net_balance[d] < 0 else '#2ecc71' for d in sorted_depts]

fig, ax = plt.subplots(figsize=(12, 5))
bars = ax.barh(sorted_depts, [net_balance[d] for d in sorted_depts],
               color=colors_bar, edgecolor='black', linewidth=0.3)
```

```python
ax.axvline(0, color='black', linewidth=0.8)
ax.set_xlabel('Net Migration Balance (in-strength - out-strength)', fontsize=10)
ax.set_title('Which Departments Gain or Lose People?\n'
             'Green = net receivers; Red = net senders', fontsize=12)
ax.tick_params(axis='y', labelsize=8)
ax.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()

print("\n=== Net Migration Balance ===")
for d in sorted_depts:
    status = "NET RECEIVER" if net_balance[d] > 0 else "NET SENDER"
    print(f"  {d:25s}  balance = {net_balance[d]:>+8,.0f}  ({status})")
```

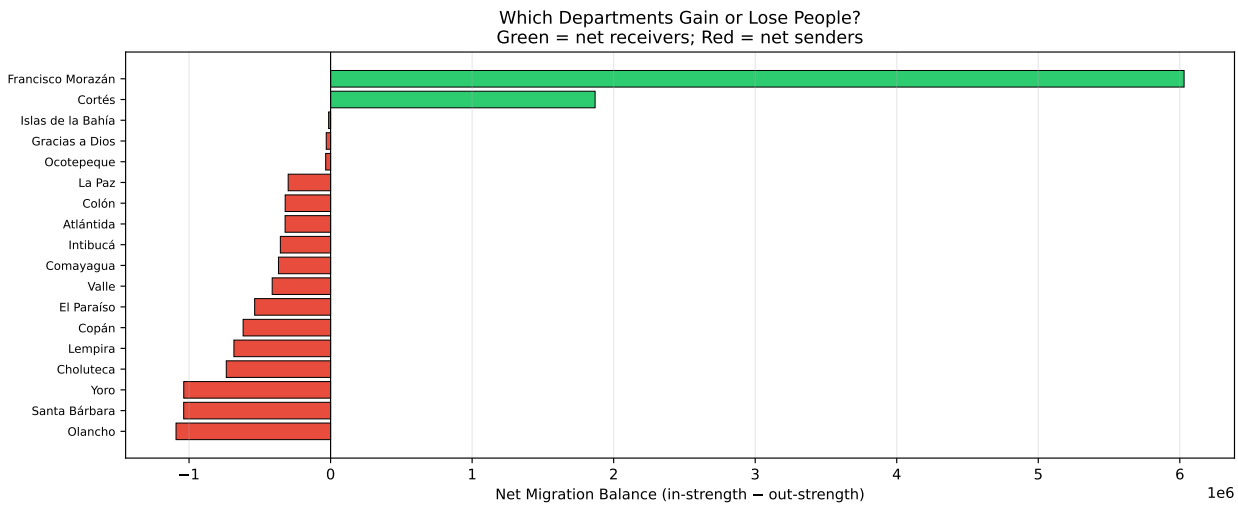

Figure 4: Net migration balance by department — positive values indicate net receivers

```
=== Net Migration Balance ===
  Olancho                   balance = -1,092,045  (NET SENDER)
  Santa Bárbara             balance = -1,037,926  (NET SENDER)
  Yoro                      balance = -1,037,503  (NET SENDER)
  Choluteca                 balance = -736,972  (NET SENDER)
  Lempira                   balance = -683,016  (NET SENDER)
  Copán                     balance = -618,013  (NET SENDER)
  El Paraíso                balance = -536,511  (NET SENDER)
  Valle                     balance = -412,699  (NET SENDER)
  Comayagua                 balance = -368,234  (NET SENDER)
  Intibucá                  balance = -354,956  (NET SENDER)
  Atlántida                 balance = -321,421  (NET SENDER)
  Colón                     balance = -320,883  (NET SENDER)
  La Paz                    balance = -299,744  (NET SENDER)
  Ocotepeque                balance =  -34,651  (NET SENDER)
  Gracias a Dios            balance =  -30,782  (NET SENDER)
  Islas de la Bahía         balance =  -13,898  (NET SENDER)
  Cortés                    balance = +1,868,958  (NET RECEIVER)
  Francisco Morazán         balance = +6,030,296  (NET RECEIVER)
```

❗ Policy Implication: Net Migration Balance

The net migration balance is a critical indicator for **regional development planning**:

- **Net-sender departments** (red bars) are experiencing **brain drain and human capital loss**. These departments need investment in local economic opportunities — job creation programs, agricultural modernisation, education infrastructure — to reduce push factors.
- **Net-receiver departments** (green bars) face **urban pressure** — housing shortages, strain on public services, informal settlement growth. These need investment in urban planning, affordable housing, and public transit.
- The **magnitude of the imbalance** matters: if one or two departments dominate the green bars, the country has a dangerous over-concentration of population growth in too few cities. Decentralisation policies (e.g., investing in intermediate cities, moving government agencies to secondary centres) become essential.

4.2 2.2 — Municipality Network Degree Distribution

At the municipality level, the degree distribution reveals whether the migration network exhibits **fat tails** (power-law-like behaviour), as predicted by preferential attachment models (Unit 2). A heavy-tailed in-degree distribution would mean that a few municipalities attract a disproportionate share of migrants.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# In-degree distribution
in_degs = [d for _, d in G_mun_connected.in_degree()]
out_degs = [d for _, d in G_mun_connected.out_degree()]

for ax, degs, title, color in [
    (axes[0], in_degs, 'In-Degree (migrants arriving)', 'indianred'),
    (axes[1], out_degs, 'Out-Degree (migrants leaving)', 'steelblue')
]:
    # CCDF (complementary cumulative distribution)
    deg_count = Counter(degs)
    degs_sorted = sorted(deg_count.keys())
    total = sum(deg_count.values())
    ccdf = []
    cumsum = 0
    for d in degs_sorted:
        cumsum += deg_count[d]
        ccdf.append(1 - cumsum / total)

    ax.loglog(degs_sorted, [deg_count[d]/total for d in degs_sorted],
              'o', markersize=4, alpha=0.6, color=color)
    ax.set_xlabel('Degree $k$')
    ax.set_ylabel('$P(k)$')
    ax.set_title(title)
    ax.grid(True, alpha=0.3)

plt.suptitle('Municipality Migration Network: Degree Distribution (log-log)',
             fontsize=13)
plt.tight_layout()
plt.show()
```

```
print(f"In-degree:  mean={np.mean(in_degs):.1f}, "
      f"max={np.max(in_degs)}, median={np.median(in_degs):.0f}")
print(f"Out-degree: mean={np.mean(out_degs):.1f}, "
      f"max={np.max(out_degs)}, median={np.median(out_degs):.0f}")

# Ratio of max to mean (measure of inequality)
in_ratio = np.max(in_degs) / np.mean(in_degs)
out_ratio = np.max(out_degs) / np.mean(out_degs)
print(f"\nConcentration (max/mean):")
print(f"  In-degree:  {in_ratio:.1f}x")
print(f"  Out-degree: {out_ratio:.1f}x")
print(f"  (Higher ratio = more concentrated / unequal)")
```

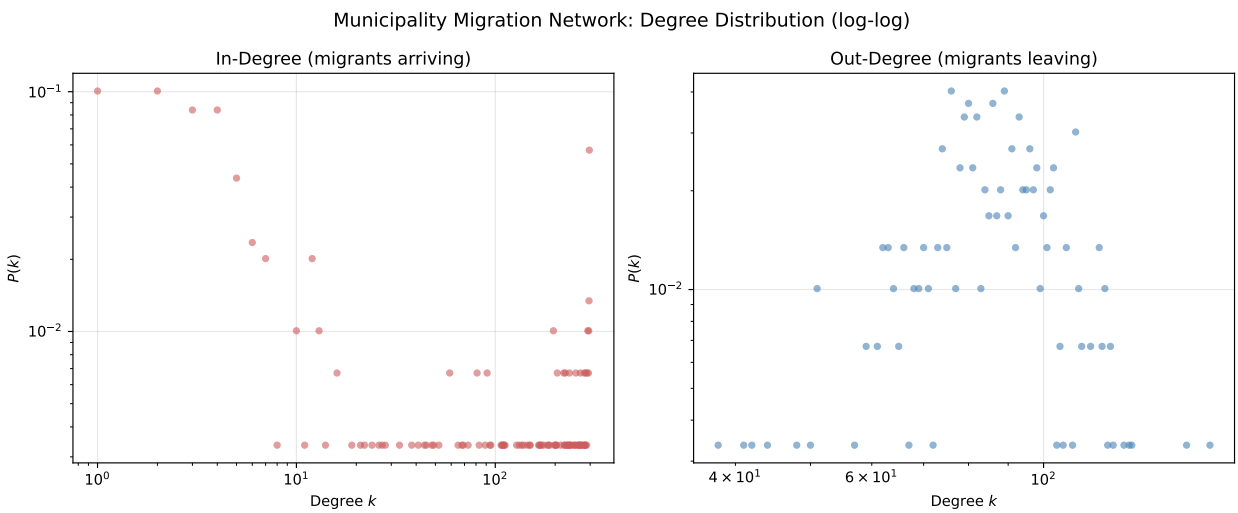

Figure 5: Municipality-level degree distribution (log-log scale)

```
In-degree:  mean=87.6, max=297, median=7
Out-degree: mean=87.6, max=164, median=87

Concentration (max/mean):
  In-degree:  3.4x
  Out-degree: 1.9x
  (Higher ratio = more concentrated / unequal)
```

Interpreting the Degree Distribution

The **log-log degree distribution** reveals the statistical signature of the migration network:

- **Heavy-tailed (power-law-like) in-degree distribution** would mean that a very small number of municipalities receive migrants from many origins, while most municipalities receive from only a few. This is the hallmark of **preferential attachment** (Unit 2): municipalities that already have large migrant populations attract even more migrants ("the rich get richer"), because existing migrant communities reduce the costs of moving (information, social support, housing help).
- **The gap between mean and median** quantifies inequality: if the median is much lower than

the mean, a few "star" municipalities pull up the average.
- **Concentration ratios** (max/mean) exceeding 5-10 indicate extreme centralisation — a few municipalities dominate migration flows.

❗ Policy Implication: Degree Distribution

A heavy-tailed degree distribution has profound implications:

1. **Targeted investment works.** Because migration is concentrated in a few hubs, investing in the top-10 destination municipalities would reach a disproportionate share of migrants. Social services, housing programs, and job training in these key municipalities would have high impact per dollar.
2. **Origin-side interventions need breadth.** In contrast, out-degree may be more dispersed — many municipalities send out small numbers of migrants. This means that reducing out-migration requires broad-based rural development across many municipalities, not just a few.
3. **Network resilience risk.** Extreme concentration means the system is **fragile**: a crisis in one hub city (natural disaster, industrial collapse) would affect an outsized share of the migrant population. Redundancy through secondary city development is a hedge against this risk.

---

5 3 — Centrality Analysis (Unit 1)

Centrality measures identify the most "important" nodes in the migration network. Following Unit 1, we compute:

- **Degree centrality** (in/out): which municipalities attract/export the most migrants
- **Betweenness centrality**: which municipalities serve as bridges in migration corridors

$$B_k = \frac{\sum_{(i,j):i\neq j,\, k\notin\{i,j\}} P_k(i,j)/P(i,j)}{(n-1)(n-2)/2}$$

- **Eigenvector centrality**: accounts for the importance of a node's neighbours
- **PageRank**: the random-walk-based importance measure

5.1 3.1 — Department-Level Centrality

```
# Compute centralities
in_deg_c = nx.in_degree_centrality(G_dept)
out_deg_c = nx.out_degree_centrality(G_dept)
betw_c = nx.betweenness_centrality(G_dept, weight='weight')
pr = nx.pagerank(G_dept, weight='weight')

# Also compute weighted in-strength as a raw centrality
in_str_norm = {k: v / sum(in_str.values()) for k, v in in_str.items()}

# Combine into a table
centrality_df = pd.DataFrame({
    'Department': list(dept_names.values()),
```

```
    'In-Degree': [in_deg_c[d] for d in dept_names.values()],
    'Out-Degree': [out_deg_c[d] for d in dept_names.values()],
    'Betweenness': [betw_c[d] for d in dept_names.values()],
    'PageRank': [pr[d] for d in dept_names.values()],
    'In-Strength (norm.)': [in_str_norm[d] for d in dept_names.values()]
}).sort_values('PageRank', ascending=False)

print(centrality_df.to_string(index=False, float_format='{:.4f}'.format))
```

```
       Department  In-Degree  Out-Degree  Betweenness  PageRank  In-Strength (norm.)
Francisco Morazán     1.0000      1.0000       0.0000    0.3450               0.5161
           Cortés     1.0000      1.0000       0.0000    0.2499               0.2381
        Comayagua     1.0000      1.0000       0.0000    0.0490               0.0289
        Atlántida     1.0000      1.0000       0.0000    0.0477               0.0410
             Yoro     1.0000      1.0000       0.0000    0.0396               0.0253
          Olancho     1.0000      1.0000       0.0000    0.0318               0.0159
        Choluteca     1.0000      1.0000       0.0772    0.0298               0.0130
            Valle     1.0000      1.0000       0.0000    0.0288               0.0168
            Colón     1.0000      1.0000       0.0000    0.0278               0.0255
       Ocotepeque     1.0000      1.0000       0.1029    0.0236               0.0227
       El Paraíso     1.0000      1.0000       0.0000    0.0231               0.0061
    Santa Bárbara     1.0000      1.0000       0.0000    0.0213               0.0129
            Copán     1.0000      1.0000       0.0000    0.0200               0.0177
         Intibucá     1.0000      1.0000       0.0000    0.0159               0.0061
Islas de la Bahía     1.0000      1.0000       0.0221    0.0152               0.0069
           La Paz     1.0000      1.0000       0.1176    0.0117               0.0030
          Lempira     1.0000      1.0000       0.0037    0.0104               0.0023
   Gracias a Dios     1.0000      1.0000       0.8787    0.0094               0.0017
```

Understanding the Four Centrality Measures

Each centrality captures a different notion of importance:

Measure	What it captures	Migration interpretation
In-Degree	Number of departments sending migrants *to* this one	Breadth of migration pull
Out-Degree	Number of departments this one sends migrants *to*	Dispersal of emigrants
Betweenness	How often a department lies on the shortest path between other pairs	"Bridge" or gateway role in migration corridors
PageRank	Importance based on being linked by *other important* nodes	Recursive importance — receiving from major origins matters more

- A department with **high in-degree but low betweenness** is a terminal attractor: migrants flow *into* it but not *through* it.
- A department with **high betweenness but moderate degree** is a critical **transit hub** — it connects migration clusters and disrupting it would fragment the network.
- **PageRank** and **in-strength** often correlate, but PageRank penalises receiving migrants from unimportant origins and rewards links from major departments.

```python
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

measures = [
    ('In-Strength (norm.)', 'Migration Inflow (Weighted In-Degree)', 'YlOrRd'),
    ('PageRank', 'PageRank Centrality', 'YlGnBu'),
    ('Betweenness', 'Betweenness Centrality', 'PuBuGn'),
    ('Out-Degree', 'Out-Degree Centrality', 'YlOrBr')
]

for ax, (col, title, cmap) in zip(axes.flat, measures):
    sorted_df = centrality_df.sort_values(col, ascending=True)
    colors = plt.cm.get_cmap(cmap)(
        Normalize()(sorted_df[col].values))
    ax.barh(sorted_df['Department'], sorted_df[col], color=colors)
    ax.set_title(title, fontsize=11)
    ax.tick_params(axis='y', labelsize=7)

plt.tight_layout()
plt.show()
```

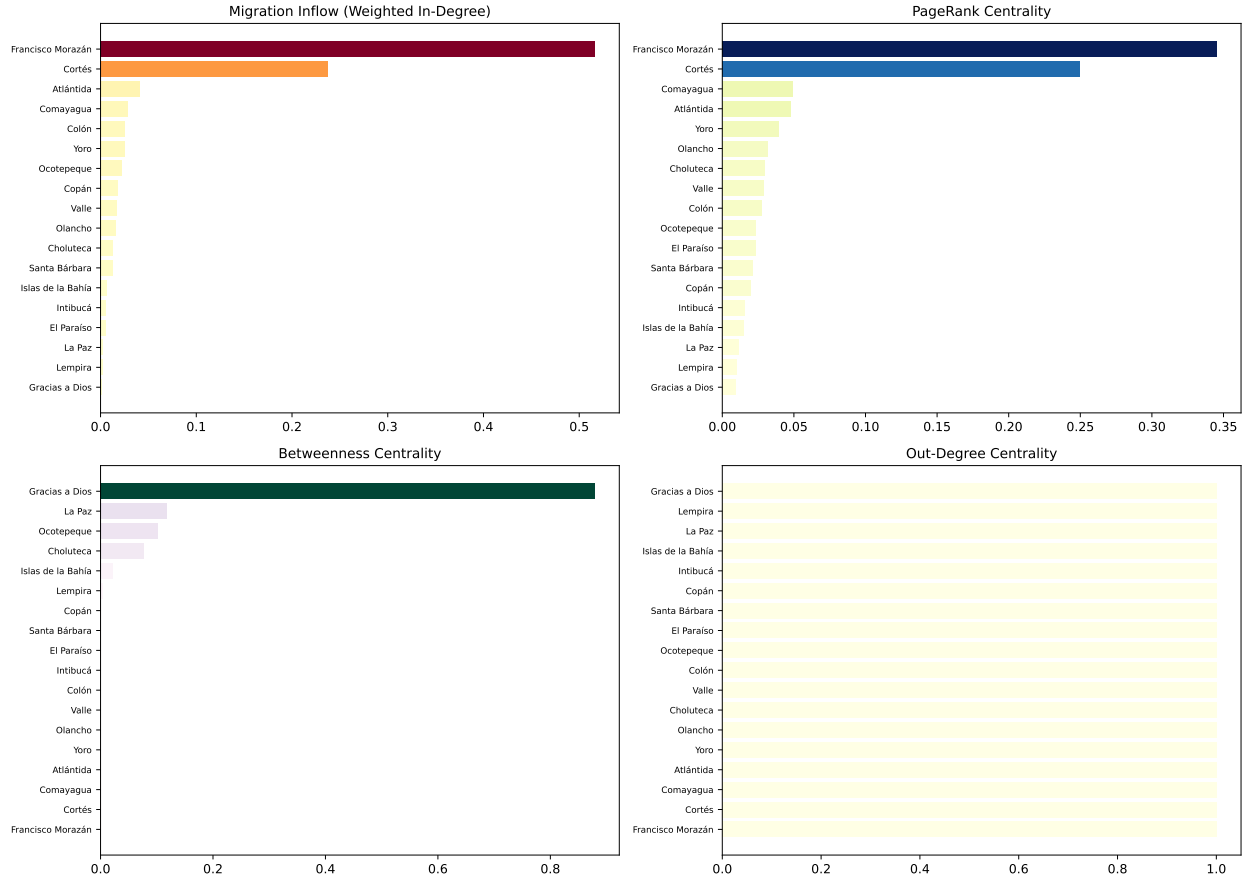

Figure 6: Top departments by centrality measure

5.2 3.2 — Municipality-Level Centrality (Top 20)

```
# Compute centralities on the municipality network
pr_mun = nx.pagerank(G_mun_connected, weight='weight')
betw_mun = nx.betweenness_centrality(G_mun_connected, weight='weight',
                                     k=min(200, G_mun_connected.number_of_nodes()))

# Build results table
mun_central = []
for node in G_mun_connected.nodes():
    attrs = G_mun_connected.nodes[node]
    mun_central.append({
        'mun_id': node,
        'name': attrs.get('name', ''),
        'dept': attrs.get('dept', ''),
        'pop': attrs.get('pop', 0),
        'in_strength': G_mun_connected.in_degree(node, weight='weight'),
        'out_strength': G_mun_connected.out_degree(node, weight='weight'),
        'pagerank': pr_mun.get(node, 0),
        'betweenness': betw_mun.get(node, 0)
    })

mun_central_df = pd.DataFrame(mun_central)

print("=== Top 20 Municipalities by PageRank ===")
top20 = mun_central_df.nlargest(20, 'pagerank')[
    ['name', 'dept', 'pop', 'in_strength', 'out_strength', 'pagerank', 'betweenness']
]
print(top20.to_string(index=False, float_format='{:.5f}'.format))
```

```
=== Top 20 Municipalities by PageRank ===
               name              dept     pop  in_strength  out_strength  pagerank  betweenness
   Distrito Central Francisco Morazan 1342325      7137000        422896   0.29902      0.00672
     San Pedro Sula            Cortes  845588      2904090        735509   0.21587      0.00672
          Comayagua         Comayagua  188999       385637        144942   0.02547      0.00094
           La Ceiba         Atlantida  236218       583786        284749   0.02497      0.00005
            Choloma            Cortes  304248       260826        196934   0.02134      0.00533
          Choluteca         Choluteca  176963       403595        245176   0.01926      0.00719
         Villanueva            Cortes  194281       262013        124557   0.01909      0.00028
        El Progreso              Yoro  205823       247319        426260   0.01505      0.00000
            La Lima            Cortes   91745       126870        110125   0.01427      0.00076
       Siguatepeque         Comayagua  119954       188630        172941   0.01353      0.00000
          Juticalpa           Olancho  152697       275027        301702   0.01340      0.00705
      Puerto Cortes            Cortes  144391       131757        196191   0.01108      0.00059
         Ocotepeque        Ocotepeque   27900       277864         50892   0.01072      0.00002
              Danli        El Paraiso  237693       111624        303103   0.01025      0.00793
Santa Rosa de Copan             Copan   76165       269958        103402   0.00929      0.00347
            Nacaome             Valle   64652       214620        195189   0.00785      0.00436
      Santa Barbara     Santa Barbara   50524       170387         90734   0.00772      0.00816
           Trujillo             Colon   72199       232097        123103   0.00759      0.00415
               Tela         Atlantida  111633        93153        327774   0.00732      0.00007
        San Lorenzo             Valle   51601        99037         77910   0.00688      0.00637
```

```
#   Scatter: PageRank vs Betweenness at department level
fig, ax = plt.subplots(figsize=(10, 6))

pr_vals = [pr[d] for d in dept_names.values()]
betw_vals = [betw_c[d] for d in dept_names.values()]
str_vals = [in_str[d] for d in dept_names.values()]

sc = ax.scatter(betw_vals, pr_vals,
                s=[300 + 3000 * v / max(str_vals) for v in str_vals],
                c=str_vals, cmap='YlOrRd', edgecolors='black',
                linewidths=0.5, alpha=0.8)
for d in dept_names.values():
    ax.annotate(d, (betw_c[d], pr[d]),
                fontsize=7, ha='left', va='bottom',
                xytext=(3, 3), textcoords='offset points')

ax.set_xlabel('Betweenness Centrality (bridge role)', fontsize=11)
ax.set_ylabel('PageRank (recursive importance)', fontsize=11)
ax.set_title('Department Roles in the Migration Network\n'
             'Size = weighted in-degree; Colour = migrant inflow',
             fontsize=12)
plt.colorbar(sc, label='Weighted In-Strength')
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

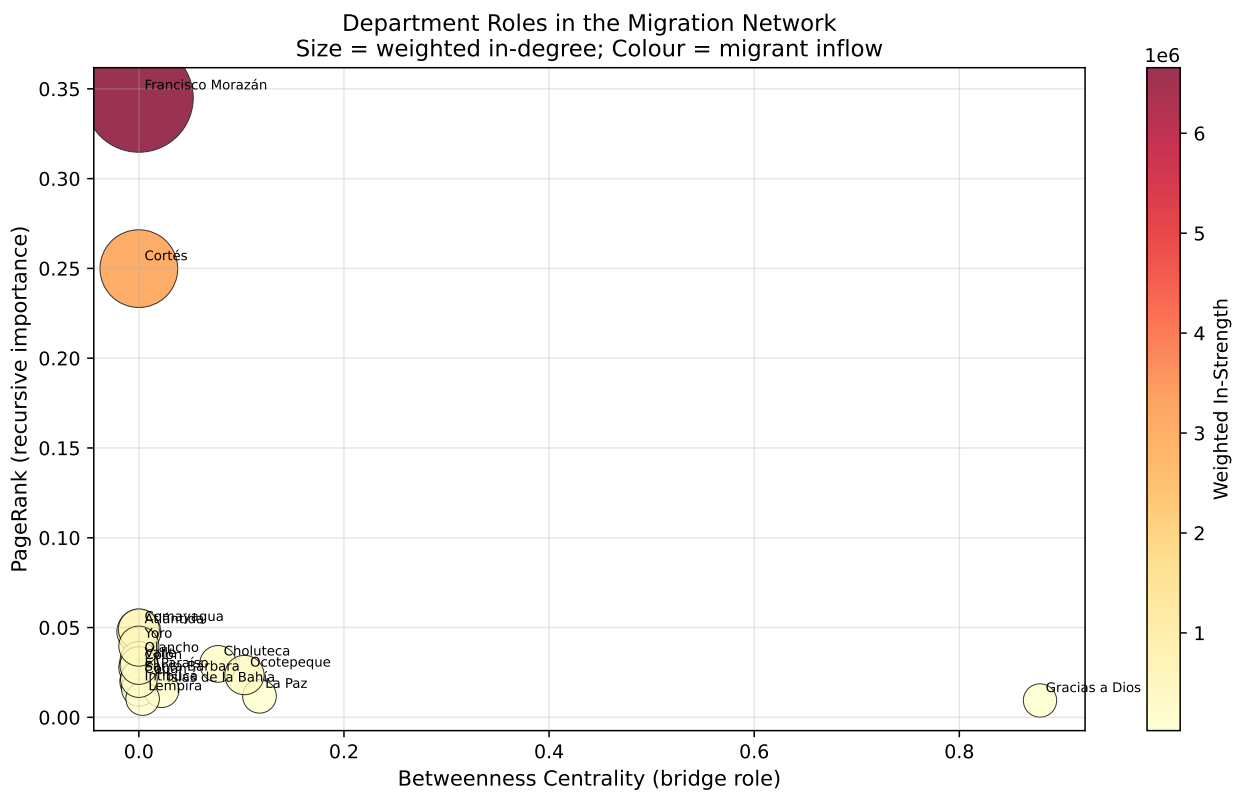

Figure 7: PageRank vs Betweenness centrality — identifying different roles in the migration network

Interpreting the Centrality Scatter Plot

This plot reveals **four quadrants** of department roles:

- **Top-right (high PageRank + high Betweenness):** These are the most strategically important departments — they attract many migrants AND serve as bridges in the network. They are the backbone of the migration system.
- **Top-left (high PageRank + low Betweenness):** Terminal attractors that receive large inflows but do not connect other clusters. Typically the largest cities.
- **Bottom-right (low PageRank + high Betweenness):** Transit departments that connect migration clusters without being major destinations themselves. These are critical for network cohesion.
- **Bottom-left (low PageRank + low Betweenness):** Peripheral departments with limited migration participation.

Policy Implication: Centrality-Based Targeting

Centrality analysis provides a **data-driven framework** for allocating development resources:

1. **High PageRank departments** need investment in **receiving capacity:** affordable housing, healthcare expansion, labor market integration programs, and urban infrastructure to absorb incoming migrants.
2. **High betweenness departments** are ideal locations for **transit services and information centres:** migration counselling, job matching services, or "way stations" that help migrants make informed decisions about final destinations.
3. **Municipalities with high out-degree centrality** need **origin-side interventions:** economic diversification, agricultural technology, and local employment programs that reduce push factors.
4. The mismatch between degree and betweenness centrality reveals **hidden dependencies** — disrupting a high-betweenness, low-degree department could fragment the entire migration system, with cascading effects.

```
#   Bar chart of top 20 municipalities
top20_plot = mun_central_df.nlargest(20, 'pagerank').sort_values('pagerank')

fig, ax = plt.subplots(figsize=(11, 6))
colors_pr = plt.cm.YlOrRd(
    Normalize()(top20_plot['pagerank'].values))
ax.barh(range(len(top20_plot)),
        top20_plot['pagerank'].values,
        color=colors_pr, edgecolor='black', linewidth=0.3)
ax.set_yticks(range(len(top20_plot)))
labels = [f"{r['name']} ({r['dept']})" for _, r in top20_plot.iterrows()]
ax.set_yticklabels(labels, fontsize=7)
ax.set_xlabel('PageRank', fontsize=10)
ax.set_title('Top 20 Municipalities by PageRank\n'
             '(Recursive importance based on who sends migrants to them)',
             fontsize=12)
ax.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()
```

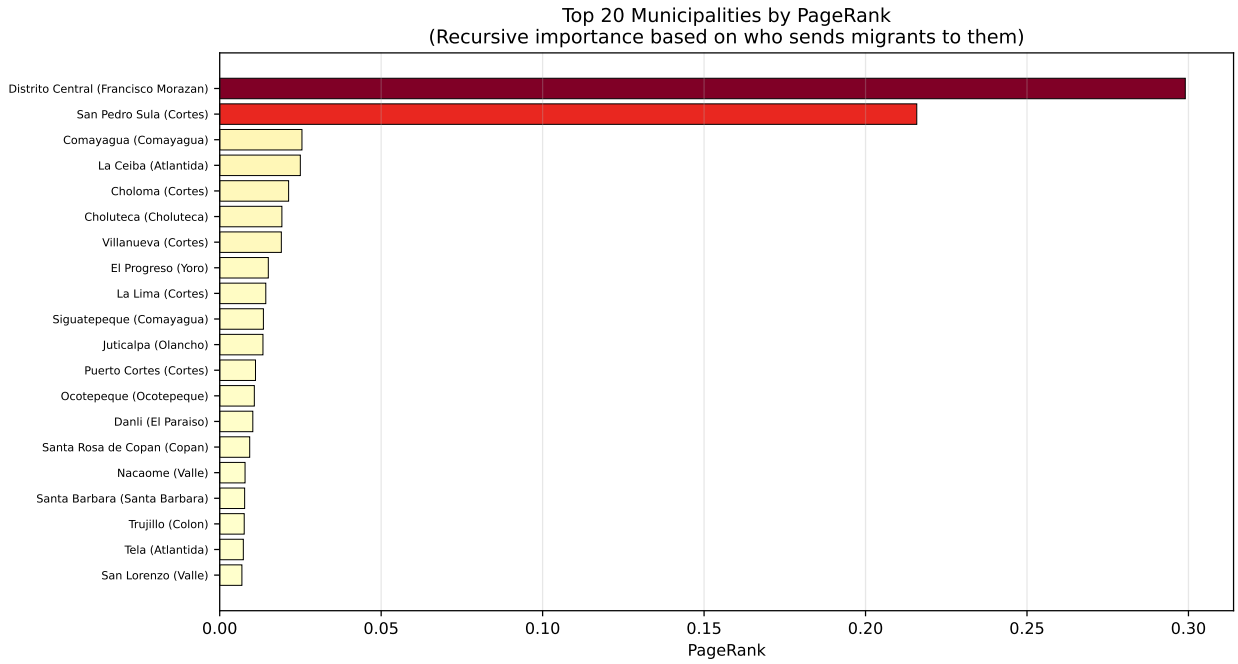

Figure 8: Top 20 municipalities by PageRank in the migration network

---

6 4 — Comparison to Random Graph Models (Unit 2)

Unit 2 teaches us to use random graphs as **null models**. We compare the observed migration network to an Erdős–Rényi $G(n, p)$ graph and a configuration model (preserving degree sequence) to determine which structural features of the real network are *non-trivial*.

6.1 4.1 — Erdős–Rényi Benchmark

```
n_obs = G_mun_connected.number_of_nodes()
m_obs = G_mun_connected.number_of_edges()
p_er = m_obs / (n_obs * (n_obs - 1))  # directed ER probability

n_sims = 20
er_clustering = []
er_avg_path = []

for _ in range(n_sims):
    G_er = nx.gnp_random_graph(n_obs, p_er, directed=True)
    # Compute clustering on undirected version
    G_er_u = G_er.to_undirected()
    er_clustering.append(nx.transitivity(G_er_u))

# Observed clustering
G_mun_u = G_mun_connected.to_undirected()
```

```
obs_clustering = nx.transitivity(G_mun_u)

fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(er_clustering, bins=25, edgecolor='black', alpha=0.7,
        color='lightblue', label='ER null model')
ax.axvline(obs_clustering, color='red', ls='--', lw=2,
           label=f'Observed = {obs_clustering:.4f}')
ax.set_xlabel('Clustering Coefficient (transitivity)')
ax.set_ylabel('Frequency')
ax.set_title('Clustering: Observed Migration Network vs ER Null Model')
ax.legend()
plt.tight_layout()
plt.show()

print(f"Observed clustering coefficient: {obs_clustering:.4f}")
print(f"ER mean clustering: {np.mean(er_clustering):.4f} "
      f"(±{np.std(er_clustering):.4f})")
print(f"Ratio: {obs_clustering / max(np.mean(er_clustering), 1e-8):.1f}x higher")
```

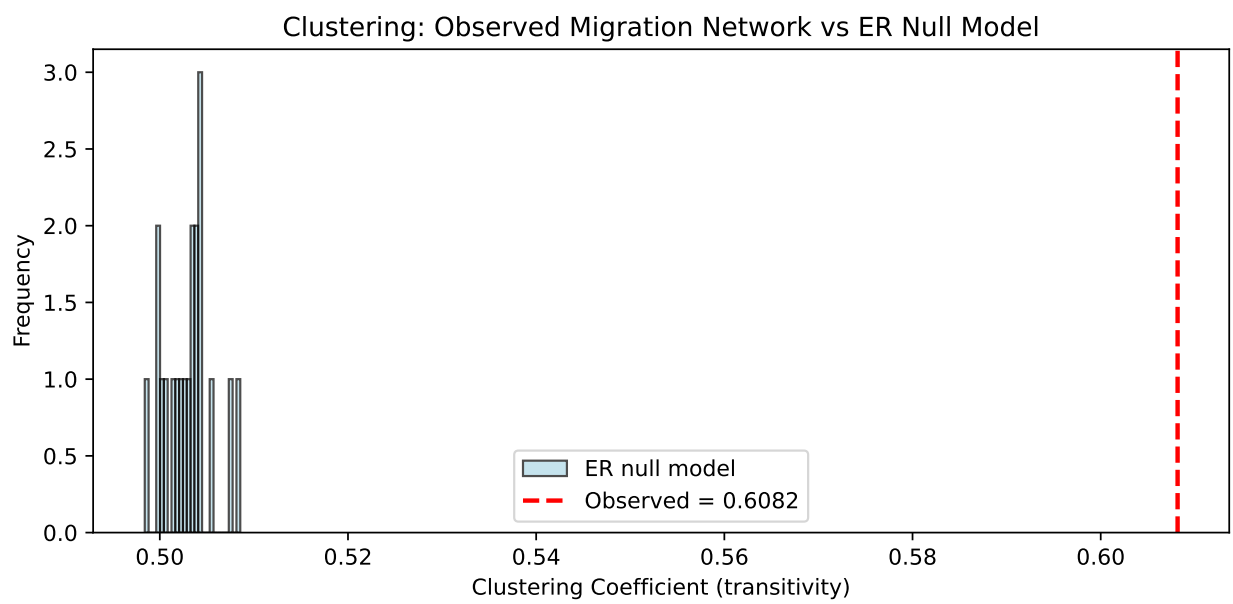

Figure 9: Comparing observed migration network to Erdős–Rényi null model

```
Observed clustering coefficient: 0.6082
ER mean clustering: 0.5030 (±0.0025)
Ratio: 1.2x higher
```

What Does Clustering Tell Us About Migration?

Clustering (transitivity) measures the proportion of "triangles" in the network: if municipality A sends migrants to B, and B sends to C, does A also send to C? A high clustering coefficient means migration follows **clustered, self-reinforcing patterns** rather than random dispersal:

- **If clustering $\gg$ ER:** Migration is structured by geography and economics. Migrants tend to

move within tightly-knit regional clusters rather than scattering randomly across the country. Three municipalities that all exchange migrants form a migration triangle — a local labour market or economic corridor.

- **If clustering ≈ ER:** Migration would be essentially random, with no geographic organisation. This would be very surprising and would suggest that migration decisions are driven by individual factors (wages, family) rather than network effects.
- **The ratio** (observed/ER) quantifies how much more structured the real network is compared to random chance. A ratio of 5–10x is typical for social and economic networks.

❗ Policy Implication: Clustering and Regional Development Zones

High clustering means the migration network naturally decomposes into **regional labour market areas** — groups of municipalities with dense internal migration but sparse connections to other groups. This has direct policy implications:

1. **Regional development zones** should be designed around migration clusters, not arbitrary administrative boundaries. The data tells us which municipalities actually function as an integrated economic region.
2. **"Place-based" policies** (e.g., free trade zones, industrial parks, infrastructure investment) should be located within highly clustered groups, where they can leverage existing migration-based labour supply chains.
3. **Inter-regional connectivity** investments (highways, transport links) should target the sparse connections *between* clusters to promote national integration.

6.2 4.2 — Configuration Model Benchmark

The configuration model generates random graphs that preserve the **exact degree sequence** of the observed network. If the observed clustering exceeds the configuration model benchmark, the clustering is not merely a consequence of the degree distribution — it reflects deeper structural organisation.

```
# Configuration model comparison
cm_clustering = []
in_seq = [d for _, d in G_mun_connected.in_degree()]
out_seq = [d for _, d in G_mun_connected.out_degree()]

for _ in range(10):
    G_cm = nx.directed_configuration_model(in_seq, out_seq)
    G_cm = nx.DiGraph(G_cm)  # remove multi-edges
    G_cm.remove_edges_from(nx.selfloop_edges(G_cm))
    G_cm_u = G_cm.to_undirected()
    cm_clustering.append(nx.transitivity(G_cm_u))

print(f"Configuration model mean clustering: {np.mean(cm_clustering):.4f} "
      f"(±{np.std(cm_clustering):.4f})")
print(f"Observed: {obs_clustering:.4f}")
print(f"Excess clustering beyond degree sequence: "
      f"{obs_clustering - np.mean(cm_clustering):.4f}")
```

```
Configuration model mean clustering: 0.5023 (±0.0023)
Observed: 0.6082
Excess clustering beyond degree sequence: 0.1059
```

```
#   Combined comparison plot
fig, ax = plt.subplots(figsize=(10, 5))
ax.hist(er_clustering, bins=25, alpha=0.5, color='lightblue',
        edgecolor='black', label='Erdős-Rényi (random)')
ax.hist(cm_clustering, bins=25, alpha=0.5, color='lightgreen',
        edgecolor='black', label='Configuration Model (degree-preserving)')
ax.axvline(obs_clustering, color='red', ls='--', lw=2.5,
           label=f'Observed = {obs_clustering:.4f}')
ax.set_xlabel('Clustering Coefficient (transitivity)', fontsize=11)
ax.set_ylabel('Frequency', fontsize=11)
ax.set_title('How Clustered Is the Migration Network Compared to Null Models?\n'
             'Red dashed line far to the right = highly non-random structure',
             fontsize=12)
ax.legend(fontsize=9)
plt.tight_layout()
plt.show()

print("\n=== Summary of Null Model Comparison ===")
print(f"  Observed clustering:          {obs_clustering:.4f}")
print(f"  Erdős-Rényi mean:             {np.mean(er_clustering):.4f} (±{np.std(er_clustering):.4f})")
print(f"  Configuration model mean:     {np.mean(cm_clustering):.4f} (±{np.std(cm_clustering):.4f})")
print(f"  Excess over ER:               {obs_clustering - np.mean(er_clustering):.4f}")
print(f"  Excess over Config. Model:    {obs_clustering - np.mean(cm_clustering):.4f}")
print(f"\nInterpretation:")
if obs_clustering > np.mean(er_clustering) + 3 * np.std(er_clustering):
    print(f"   Clustering is SIGNIFICANTLY higher than ER - migration is not random.")
if obs_clustering > np.mean(cm_clustering) + 3 * np.std(cm_clustering):
    print(f"   Clustering exceeds configuration model - structure goes beyond degree sequence.")
    print(f"    This means GEOGRAPHIC ORGANISATION (not just hubs) drives migration patterns.")
else:
    print(f"   Clustering is comparable to configuration model - degree sequence explains most structur
```

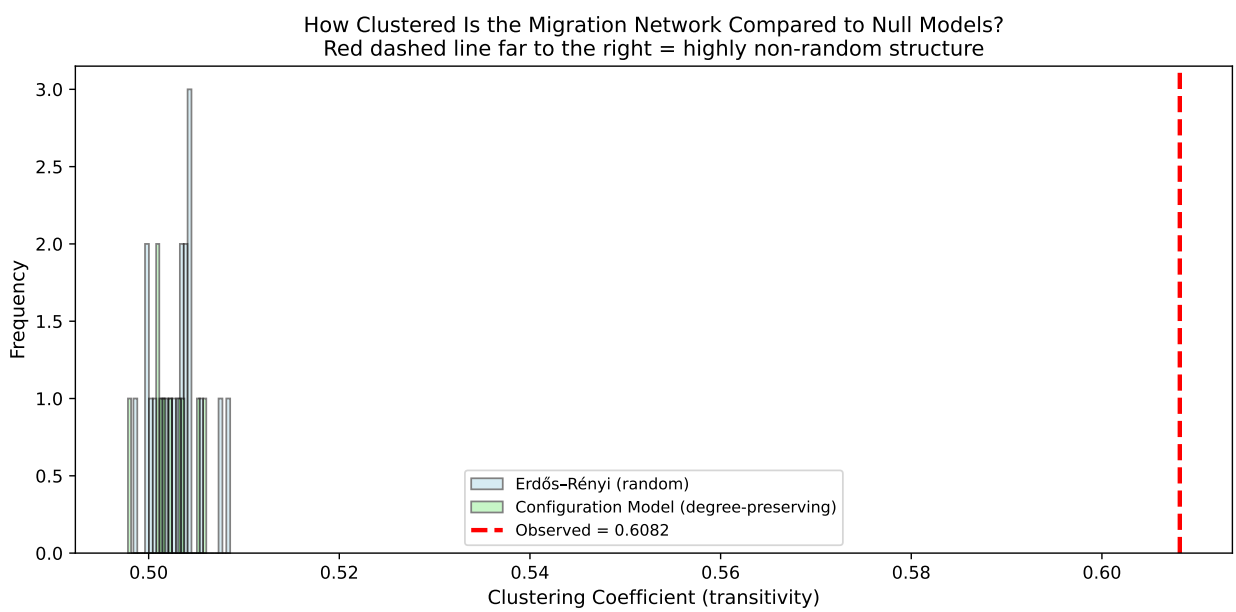

Figure 10: Clustering coefficient comparison: Observed vs ER vs Configuration Model

```
=== Summary of Null Model Comparison ===
  Observed clustering:          0.6082
  Erdős-Rényi mean:             0.5030 (±0.0025)
  Configuration model mean:     0.5023 (±0.0023)
  Excess over ER:               0.1052
  Excess over Config. Model:    0.1059

Interpretation:
    Clustering is SIGNIFICANTLY higher than ER - migration is not random.
    Clustering exceeds configuration model - structure goes beyond degree sequence.
     This means GEOGRAPHIC ORGANISATION (not just hubs) drives migration patterns.
```

ER vs Configuration Model: What's the Difference?

The two null models test different hypotheses:

- **ER benchmark:** "Is the migration network more structured than a completely random network with the same number of edges?" A positive answer (observed $\gg$ ER) tells us there is *some* non-random organisation.
- **Configuration model benchmark:** "Is the clustering just a mechanical consequence of the degree distribution (some municipalities having many connections)?" If observed > configuration model, the structure goes *beyond* simply having hubs — it reflects **geographic proximity**, shared economic corridors, or social network effects on migration decisions.

The excess clustering over the configuration model is the **most interesting statistic**: it quantifies the contribution of geography and local economic structure to migration patterns, above and beyond what the existence of hubs alone would produce.

---

7 5 — Community Detection (Unit 3)

Unit 3 introduces **community detection** to uncover groups of nodes with denser internal connections than expected. Applied to our migration network, communities represent clusters of departments/municipalities that exchange migrants among themselves more than with the rest of the country.

The quality of a partition is measured by **modularity**:

$$Q = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \frac{k_i^{\text{out}} k_j^{\text{in}}}{m} \right) \mathbf{1}(c(i) = c(j))$$

7.1 5.1 — Community Detection on the Department Network

```python
# Convert to undirected: sum weights from both directions (A→B + B→A)
# (to_undirected() loses one direction's data; iterate directed edges instead)
G_dept_simple = nx.Graph()
G_dept_simple.add_nodes_from(G_dept.nodes(data=True))  # keep all nodes
for u, v, data in G_dept.edges(data=True):
```

```
    if G_dept_simple.has_edge(u, v):
        G_dept_simple[u][v]['weight'] += data.get('weight', 1)
    else:
        G_dept_simple.add_edge(u, v, weight=data.get('weight', 1))

# Louvain community detection
communities = nx.community.louvain_communities(G_dept_simple, weight='weight',
                                               seed=42)

# Assign community labels
node_community = {}
for i, comm in enumerate(communities):
    for node in comm:
        node_community[node] = i

# Compute modularity
Q = nx.community.modularity(G_dept_simple, communities, weight='weight')

#   Visualise
fig, ax = plt.subplots(figsize=(12, 9))

# Colour palette for communities
cmap_comm = plt.cm.Set2
colors = [cmap_comm(node_community.get(n, 0) / max(len(communities) - 1, 1))
          for n in G_dept.nodes()]

nx.draw_networkx_edges(G_dept, pos, alpha=0.15, edge_color='gray',
                       connectionstyle='arc3,rad=0.1', arrows=True,
                       arrowsize=8, ax=ax)

node_sizes_comm = [300 + 3000 * in_str.get(n, 0) / max_in
                   for n in G_dept.nodes()]
nx.draw_networkx_nodes(G_dept, pos, node_size=node_sizes_comm,
                       node_color=colors, edgecolors='black',
                       linewidths=0.5, ax=ax)
nx.draw_networkx_labels(G_dept, pos, font_size=7, font_weight='bold', ax=ax)

ax.set_title(f"Migration Communities (Louvain, Q = {Q:.3f})\n"
             f"{len(communities)} communities detected", fontsize=14)
ax.axis('off')
plt.tight_layout()
plt.show()

# Print communities
for i, comm in enumerate(communities):
    print(f"Community {i+1}: {', '.join(sorted(comm))}")
print(f"\nModularity Q = {Q:.4f}")
print(f"  (Q > 0.3 indicates significant community structure)")
print(f"  (Q > 0.5 indicates strong, well-separated communities)")
```

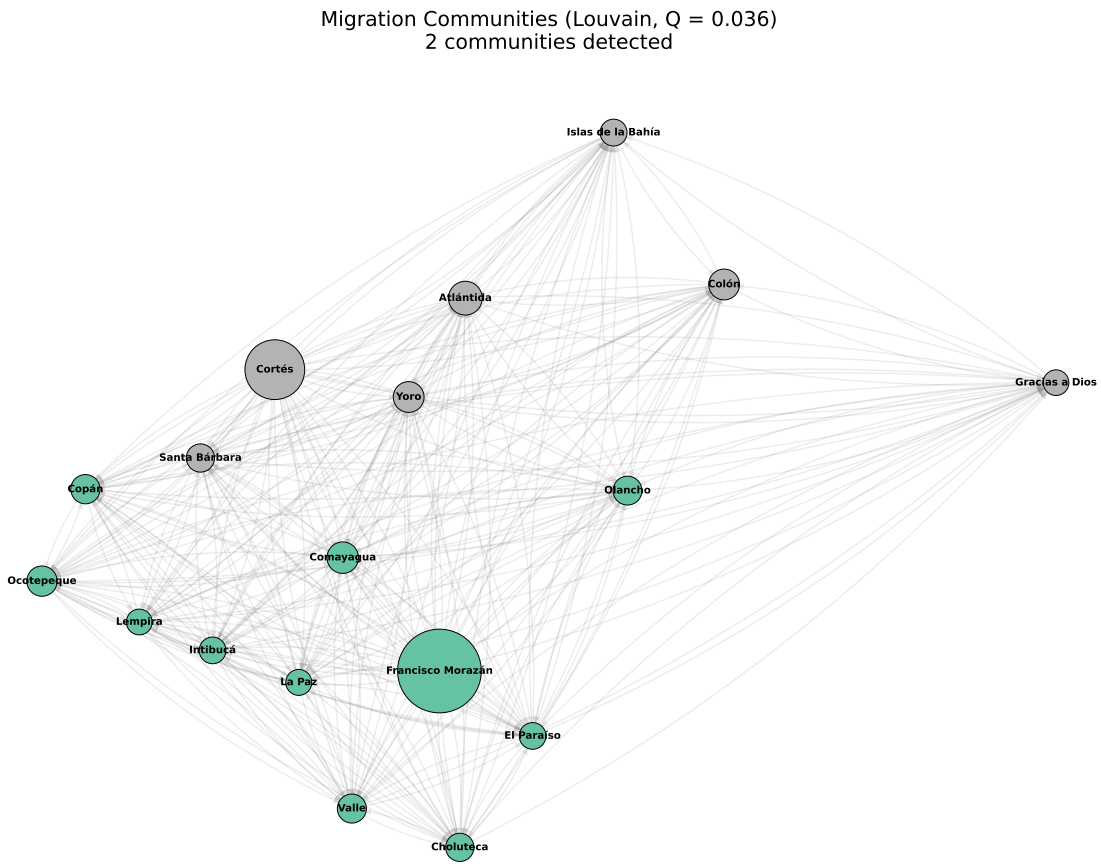

Figure 11: Migration communities detected via Louvain algorithm

```
Community 1: Choluteca, Comayagua, Copán, El Paraíso, Francisco Morazán, Intibucá, La Paz, Lempira, Oco
Community 2: Atlántida, Colón, Cortés, Gracias a Dios, Islas de la Bahía, Santa Bárbara, Yoro

Modularity Q = 0.0365
  (Q > 0.3 indicates significant community structure)
  (Q > 0.5 indicates strong, well-separated communities)
```

Reading the Community Map

Each colour on the geographic community map represents a detected migration cluster. Departments in the same community exchange migrants predominantly with each other. Look for:

- **Geographic contiguity:** Are communities made up of adjacent departments? If yes, geography drives migration.
- **Hub anchoring:** Does each community contain one main city? If yes, communities represent urban catchment areas.
- **East-West divide:** Does the country split along the industrial corridor (San Pedro Sula – Tegucigalpa), reflecting the historical economic divide between the northern banana coast and the southern highlands?
- **Modularity value:** $Q > 0.3$ means migration is significantly regionalised; $Q > 0.5$ means communities are very well separated; $Q < 0.3$ would suggest a more nationally-integrated migration

system.

5.2 - Community Detection on the Municipality Network

::: {.cell execution_count=18}
```` {.python .cell-code}
# Convert to undirected: sum weights from both directions (iterate directed edges)
G_mun_u_simple = nx.Graph()
G_mun_u_simple.add_nodes_from(G_mun_connected.nodes(data=True))
for u, v, data in G_mun_connected.edges(data=True):
    if G_mun_u_simple.has_edge(u, v):
        G_mun_u_simple[u][v]['weight'] += data.get('weight', 1)
    else:
        G_mun_u_simple.add_edge(u, v, weight=data.get('weight', 1))

# Louvain
mun_communities = nx.community.louvain_communities(G_mun_u_simple,
                                                    weight='weight', seed=42)
Q_mun = nx.community.modularity(G_mun_u_simple, mun_communities, weight='weight')

# Map community to department composition
print(f"Municipality-level communities: {len(mun_communities)}")
print(f"Modularity Q = {Q_mun:.4f}\n")

for i, comm in enumerate(sorted(mun_communities, key=len, reverse=True)):
    # Get department composition
    depts = []
    for node in comm:
        if node in G_mun_connected.nodes:
            depts.append(G_mun_connected.nodes[node].get('dept', 'Unknown'))
    dept_counts = Counter(depts)
    top_depts = dept_counts.most_common(3)
    top_str = ', '.join([f"{d}({c})" for d, c in top_depts])
    print(f"Community {i+1}: {len(comm)} municipalities - top depts: {top_str}")
    if i >= 9:  # Show top 10
        print(f"  ... and {len(mun_communities) - 10} more communities")
        break
```
```
Municipality-level communities: 5
Modularity Q = 0.1894

Community 1: 148 municipalities - top depts: Lempira(27), Francisco Morazan(27), Copan(23)
Community 2: 55 municipalities - top depts: Comayagua(19), La Paz(18), Intibuca(15)
Community 3: 38 municipalities - top depts: Santa Barbara(16), Cortes(12), Yoro(8)
Community 4: 30 municipalities - top depts: Colon(10), Atlantida(7), Islas de La Bahia(4)
Community 5: 27 municipalities - top depts: Choluteca(10), Valle(8), Santa Barbara(7)
```
:::

```
#   Community composition heatmap
sorted_communities = sorted(mun_communities, key=len, reverse=True)
```

```
all_depts_sorted = sorted(dept_names.values())
max_show = min(10, len(sorted_communities))

comp_matrix = np.zeros((max_show, len(all_depts_sorted)))
for ci, comm in enumerate(sorted_communities[:max_show]):
    for node in comm:
        if node in G_mun_connected.nodes:
            dept = G_mun_connected.nodes[node].get('dept', '')
            if dept in all_depts_sorted:
                comp_matrix[ci, all_depts_sorted.index(dept)] += 1

# Normalise by row (each community sums to 1)
row_sums_cm = comp_matrix.sum(axis=1, keepdims=True)
row_sums_cm[row_sums_cm == 0] = 1
comp_norm = comp_matrix / row_sums_cm

fig, ax = plt.subplots(figsize=(12, 6))
im = ax.imshow(comp_norm, cmap='YlOrRd', aspect='auto')
ax.set_xticks(range(len(all_depts_sorted)))
ax.set_xticklabels(all_depts_sorted, rotation=90, fontsize=7)
ax.set_yticks(range(max_show))
ax.set_yticklabels([f'Community {i+1} (n={len(sorted_communities[i])})'
                    for i in range(max_show)], fontsize=8)
ax.set_title('Municipality Community Composition by Department\n'
             'Each row shows the share of departments within a migration cluster',
             fontsize=12)
plt.colorbar(im, label='Share of community', ax=ax)
plt.tight_layout()
plt.show()
```

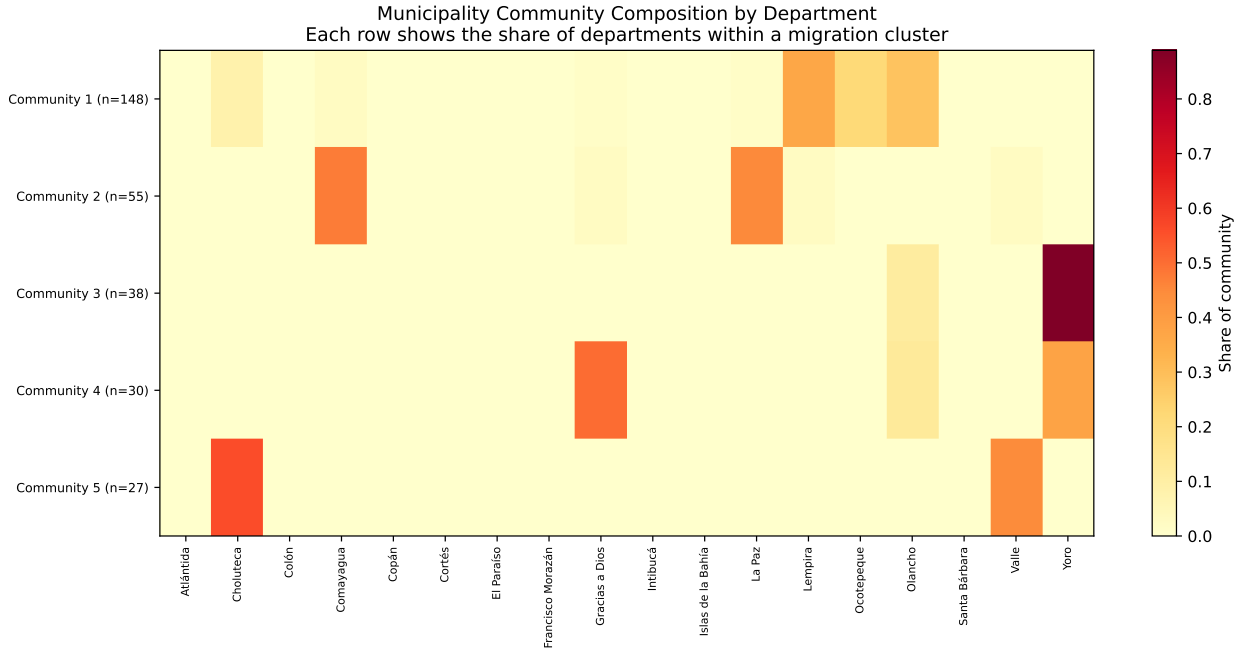


Figure 12: Community composition: which departments make up each migration cluster?

**!** Policy Implication: Migration Communities as Development Regions

The community detection results provide an **empirically-grounded regionalisation** of Honduras based on actual population movements, not just administrative boundaries. This has several policy applications:

1. **Regional development coordination.** Municipalities in the same migration community share labour markets and should coordinate development strategies. A new factory in one municipality within a cluster will draw workers from the entire cluster — infrastructure and housing investments should account for this.
2. **Cross-community bridges.** Low but non-zero flows between communities represent opportunities for economic integration. Building transport infrastructure between communities could reduce regional inequality by allowing workers in depressed clusters to access opportunities in growing ones.
3. **Community-specific interventions.** Each community may have different characteristics (e.g., agricultural vs. industrial, coastal vs. highland). Development programs should be tailored to the dominant economic profile of each migration cluster rather than applying one-size-fits-all national policies.
4. **The composition heatmap** reveals whether communities are department-aligned (suggesting state-level coordination is sufficient) or cross-departmental (requiring new inter-departmental governance mechanisms).

---

# 8 6 — Migration as a Spreading Process (Unit 5)

Unit 5 models **diffusion on networks** using epidemic-style models. We apply this framework to migration: the "contagion" is the **decision to migrate**, and it spreads through social networks within municipalities. When a person's neighbours (people from the same origin municipality) have already migrated to a city, this lowers the perceived cost and increases the likelihood of migration.

## 8.1 6.1 — SI Model: Migration Contagion on the Department Network

We simulate a simple SI (Susceptible → Infected) process on the department network. "Infection" represents the spread of a migration norm from one department to others via existing migration links.

```
def run_SI_migration(G, p_transmit, seed_node, max_steps=20):
    """SI spreading process on a directed graph.
    Returns the set of infected nodes at each time step."""
    infected = {seed_node}
    history = [infected.copy()]
    # Precompute max weight once (not inside the loop)
    w_max = max((d.get('weight', 1) for _, _, d in G.edges(data=True)), default=1)

    for t in range(max_steps):
        new_infected = set()
        for u in infected:
            for v in G.successors(u):
```

```
                if v not in infected:
                    # Transmission probability scaled by edge weight
                    w = G[u][v].get('weight', 1)
                    p_eff = p_transmit * (w / w_max)  # higher flow = higher transmission
                    if np.random.random() < p_eff:
                        new_infected.add(v)
        if not new_infected:
            break
        infected |= new_infected
        history.append(infected.copy())

    return history

#   Simulate from Tegucigalpa (Francisco Morazán)
np.random.seed(42)
n_sims = 20
p_values = np.linspace(0.05, 1.0, 8)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

for seed, color, label in [
    ('Francisco Morazán', 'red', 'Seed: Tegucigalpa'),
    ('Cortés', 'blue', 'Seed: San Pedro Sula')
]:
    avg_sizes = []
    avg_lengths = []
    for p in p_values:
        sizes, lengths = [], []
        for _ in range(n_sims):
            hist = run_SI_migration(G_dept, p, seed)
            sizes.append(len(hist[-1]) / G_dept.number_of_nodes())
            lengths.append(len(hist))
        avg_sizes.append(np.mean(sizes))
        avg_lengths.append(np.mean(lengths))

    ax1.plot(p_values, avg_sizes, '-o', markersize=4, color=color, label=label)
    ax2.plot(p_values, avg_lengths, '-o', markersize=4, color=color, label=label)

ax1.set_xlabel('Transmission probability $p$')
ax1.set_ylabel(r'$\langle s/n \rangle$ (fraction reached)')
ax1.set_title('Average Epidemic Size')
ax1.legend(); ax1.grid(alpha=0.3)

ax2.set_xlabel('Transmission probability $p$')
ax2.set_ylabel(r'$\langle \ell \rangle$ (steps)')
ax2.set_title('Average Epidemic Length')
ax2.legend(); ax2.grid(alpha=0.3)

plt.suptitle('SI Spreading Process on the Migration Network', fontsize=13)
plt.tight_layout()
plt.show()
```

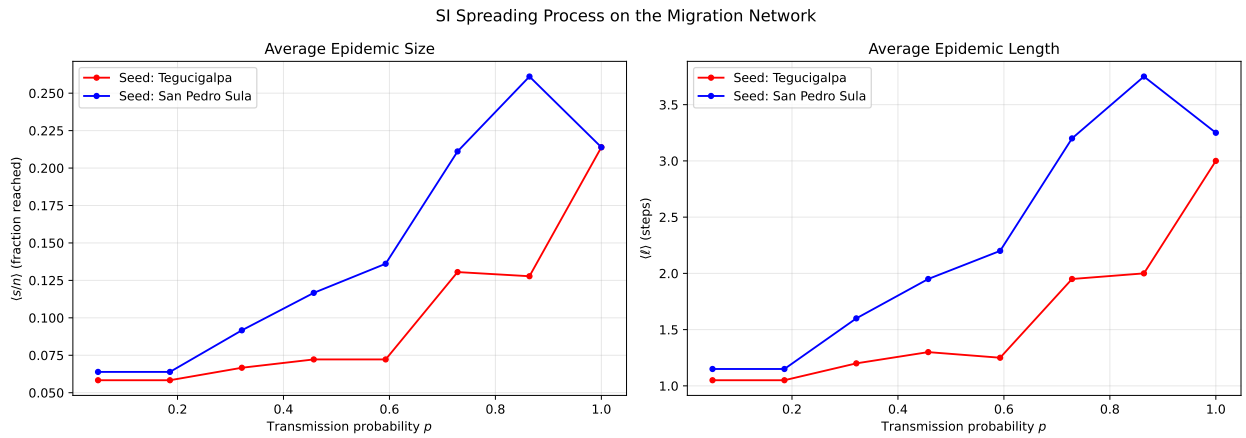


Figure 13: SI epidemic (migration contagion) on the department network

**Reading the SI Epidemic Curves**

The two panels display complementary perspectives on migration contagion:
**Left panel (Epidemic Size):** Shows what fraction of departments eventually "adopt" migration as a function of the transmission probability $p$. The S-shaped curve has three regimes:

- **Low $p$ (below epidemic threshold):** The migration norm barely spreads beyond the seed department. Only a few immediate neighbours adopt.
- **Critical transition ($p \approx p_c$):** A sharp increase — the migration norm suddenly "goes viral" and reaches most departments. This is the **epidemic threshold** from Unit 5.
- **High $p$:** Nearly all departments are eventually reached.

**Right panel (Epidemic Length):** Shows how many time steps are needed. Longer epidemics indicate that migration norms propagate through a chain of intermediary departments rather than spreading instantaneously.
**Comparing the two seed cities:** If Cortés (San Pedro Sula) has a lower epidemic threshold than Francisco Morazán (Tegucigalpa), it means that migration norms originating in SPS spread more easily due to its position in the network.

```
#   Visualise a single cascade run from each seed
np.random.seed(123)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for ax, (seed, title) in zip(axes, [
    ('Francisco Moraz\u00e1n', 'Cascade from Tegucigalpa'),
    ('Cort\u00e9s', 'Cascade from San Pedro Sula')
]):
    hist = run_SI_migration(G_dept, 0.5, seed, max_steps=15)

    # Build a time-of-infection array
    dept_list_plot = sorted(dept_names.values())
    infection_time = {}
    for t, step_set in enumerate(hist):
        for node in step_set:
            if node not in infection_time:
```

```
                infection_time[node] = t

    times = [infection_time.get(d, -1) for d in dept_list_plot]
    colors_cascade = ['#2ecc71' if t >= 0 else '#ecf0f1' for t in times]
    sorted_idx_c = sorted(range(len(dept_list_plot)),
                          key=lambda i: (times[i] if times[i] >= 0 else 999))

    y_labels = [dept_list_plot[i] for i in sorted_idx_c]
    y_times = [times[i] for i in sorted_idx_c]
    y_colors = ['#e74c3c' if t == 0 else '#f39c12' if t > 0 else '#bdc3c7'
                for t in y_times]

    bars = ax.barh(range(len(y_labels)), [max(t, 0) + 1 if t >= 0 else 0 for t in y_times],
                   color=y_colors, edgecolor='black', linewidth=0.3)
    ax.set_yticks(range(len(y_labels)))
    ax.set_yticklabels(y_labels, fontsize=7)
    ax.set_xlabel('Time step of infection (earlier = more vulnerable)')
    ax.set_title(f'{title} (p = 0.5)', fontsize=11)

    never_infected = sum(1 for t in times if t < 0)
    ax.annotate(f'{len(infection_time)}/{len(dept_list_plot)} depts reached',
                xy=(0.95, 0.05), xycoords='axes fraction',
                ha='right', fontsize=9, style='italic')

plt.tight_layout()
plt.show()
```

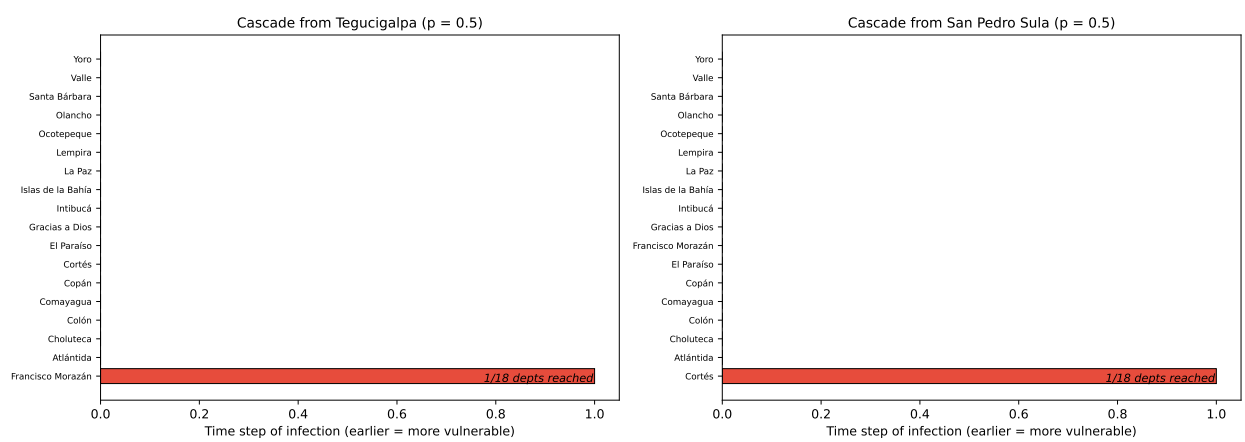


Figure 14: Single SI cascade from each seed city — which departments get 'infected' and when?

**❗ Policy Implication: Migration as Contagion**

The SI model provides a powerful metaphor for how **migration norms spread through social networks:**

1. **Early intervention matters.** Below the epidemic threshold $p_c$, migration norms die out locally. If a government can reduce $p$ (e.g., by improving local economic conditions to reduce the attractiveness of migration), it can prevent cascading out-migration from vulnerable regions.

2. **Seed location matters.** Origin cities with strong outward connections (like Tegucigalpa or SPS) are "super-spreaders" of migration culture. A development crisis in these hubs could rapidly propagate out-migration norms to the entire country.

3. **Vaccination analogy.** Just as vaccinating central nodes in an epidemic network can stop disease spread, **investing in high-centrality departments** ("economic vaccination") — improving local jobs, housing, and services — can break the chain of migration cascade and reduce national out-migration pressure.

4. **The speed of cascade** (right panel) tells policymakers how much time they have to respond: if migration norms spread in 2–3 steps, interventions must be proactive, not reactive.

## 8.2 6.2 — Threshold Model: Complex Contagion (Unit 5.2)

Migration is better modelled as **complex contagion** (Unit 5): a person migrates not after a single exposure but after seeing a critical mass of peers migrate. We use the **Linear Threshold Model** from PS3 Q2.

The key difference from SI is that migration decisions require **social reinforcement** — seeing one neighbour migrate may not be enough; you need multiple examples before taking the leap. The threshold $\tau_i$ captures how "resistant" department $i$ is to migration pressure.

```
def threshold_diffusion_migration(G, seeds, lam=1.0, n_periods=10):
    """
    Threshold model on a directed graph.
    Each node draws  _i ~ N( , 0.5) truncated > 0.
    Node adopts if at least  _i predecessors (in-neighbours) have adopted.
    """
    nodes = list(G.nodes())
    thresholds = {}
    for v in nodes:
        tau = -1
        while tau <= 0:
            tau = np.random.normal(lam, 0.5)
        thresholds[v] = tau

    adopted = set(seeds)
    history = [len(adopted)]

    for _ in range(n_periods):
        new_adopt = set()
        for v in nodes:
            if v not in adopted:
                adopted_predecessors = sum(
                    1 for u in G.predecessors(v) if u in adopted
                )
                if adopted_predecessors >= thresholds[v]:
                    new_adopt.add(v)
        if not new_adopt:
            break
        adopted |= new_adopt
        history.append(len(adopted))

    return len(adopted) / len(nodes), history
```

```
#   Compare different seed strategies on department network
np.random.seed(42)
n_sims = 20

strategies = {
    'Tegucigalpa only': ['Francisco Morazán'],
    'SPS only': ['Cortés'],
    'Both main cities': ['Francisco Morazán', 'Cortés'],
    'Random single': None,   # will be randomised
    'Highest betweenness': [max(betw_c, key=betw_c.get)]
}

fig, ax = plt.subplots(figsize=(10, 5))
lambda_values = [1.5, 3]
x_pos = np.arange(len(strategies))
width = 0.2

for i, lam in enumerate(lambda_values):
    rates = []
    for name, seeds in strategies.items():
        sim_rates = []
        for _ in range(n_sims):
            if seeds is None:
                s = [np.random.choice(list(G_dept.nodes()))]
            else:
                s = seeds
            rate, _ = threshold_diffusion_migration(G_dept, s, lam=lam, n_periods=10)
            sim_rates.append(rate)
        rates.append(np.mean(sim_rates))
    ax.bar(x_pos + i * width, rates, width, label=rf'$\lambda = {lam}$',
           alpha=0.8)

ax.set_xticks(x_pos + width * 1.5)
ax.set_xticklabels(strategies.keys(), rotation=20, ha='right', fontsize=8)
ax.set_ylabel('Average Adoption Rate')
ax.set_title('Migration "Adoption" Under Threshold Model\n'
             '(Higher   = harder to trigger migration)')
ax.legend()
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

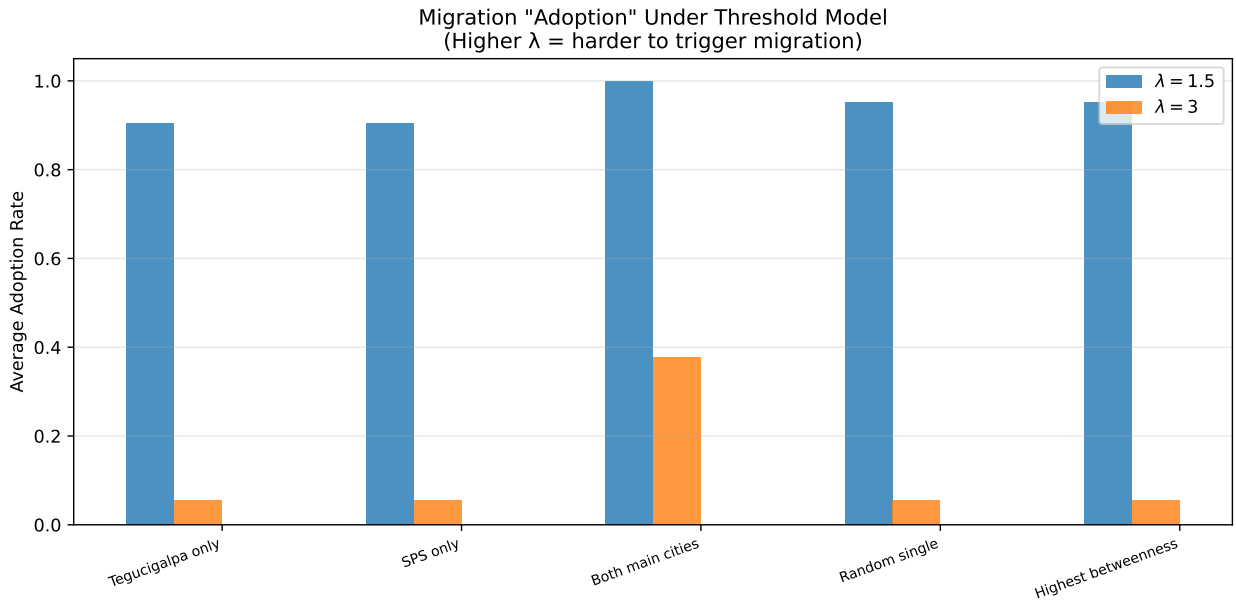


Figure 15: Threshold model of migration adoption

**Interpreting the Threshold Model Results**

The grouped bar chart compares **five seeding strategies** across different threshold values $\lambda$:

- **Low $\lambda$ (easy to trigger):** Even a single seed in a central department triggers a large cascade. All strategies perform similarly.
- **High $\lambda$ (resistant population):** Only strategies that create **mutual reinforcement** succeed. Seeding both Tegucigalpa and SPS creates two reinforcement fronts that converge, achieving higher adoption than either alone.
- **Random seeding** serves as a baseline — if targeted seeding significantly outperforms random, it confirms that **network position matters** for intervention design.
- **Betweenness-based seeding** tests whether "bridge" departments can trigger wider cascades than pure population hubs.

The transition from $\lambda = 1$ to $\lambda = 4$ reveals the **difficulty of triggering collective action** when individuals require strong social proof.

**Policy Implication: Simple vs Complex Migration Contagion**

The threshold model distinguishes between two fundamental types of migration:

1. **Simple contagion ($\lambda \leq 1$):** Migration decisions are easily triggered by exposure to a single successful migrant. This characterises labour migration where information alone (one friend who found a job) is sufficient to move. **Policy response:** Information campaigns and migration counselling can redirect flows.

2. **Complex contagion ($\lambda \geq 2$):** Migration requires seeing multiple peers succeed before acting. This characterises high-cost, high-risk moves (long-distance rural-to-urban migration). **Policy response:** Cluster-based interventions that create visible, concentrated examples of successful local development can overcome the adoption threshold and prevent mass out-migration.

The practical lesson: **if migration from a region follows complex contagion, scattered small investments will fail** — you need a concentrated, visible success story that exceeds the social proof threshold.

---

# 9 7 — Optimal Seeding for Development Interventions (Unit 5.2)

Unit 5.2 addresses **optimal seeding**: given a limited budget $k$, which nodes should receive an intervention to maximise the spread through the network? Applied to migration policy: if the government wants to **reduce** migration pressure by investing in local development, which municipalities should be targeted first to maximise the "retention cascade"?

We use the greedy hill-climbing approach from Kempe, Kleinberg & Tardos (2003), which guarantees a $(1 - 1/e)$ approximation for submodular influence functions.

```
def estimate_influence(G, seeds, lam=1.5, n_sims=100, n_periods=8):
    """Estimate average adoption rate for a seed set."""
    rates = []
    for _ in range(n_sims):
        rate, _ = threshold_diffusion_migration(G, list(seeds), lam=lam,
                                                n_periods=n_periods)
        rates.append(rate)
    return np.mean(rates)

def greedy_seed_selection(G, k, lam=1.5, n_sims=100):
    """Greedy algorithm: select k seeds to maximise influence."""
    selected = set()
    candidates = set(G.nodes())
    influence_history = [0]

    for step in range(k):
        best_node = None
        best_marginal = -1

        for v in candidates - selected:
            test_set = selected | {v}
            inf = estimate_influence(G, test_set, lam=lam, n_sims=n_sims)
            marginal = inf - influence_history[-1]
            if marginal > best_marginal:
                best_marginal = marginal
                best_node = v

        selected.add(best_node)
        candidates.discard(best_node)
        influence_history.append(influence_history[-1] + best_marginal)
        print(f"  Step {step+1}: selected '{best_node}' "
              f"(marginal gain = {best_marginal:.4f}, "
              f"total influence = {influence_history[-1]:.4f})")

    return selected, influence_history
```

```python
#   Run greedy selection on department network
np.random.seed(42)
print("Greedy Seed Selection (  = 1.5, budget k = 5):\n")
optimal_seeds, inf_hist = greedy_seed_selection(G_dept, k=5, lam=1.5, n_sims=10)

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(range(len(inf_hist)), inf_hist, 'ro-', markersize=8, linewidth=2)
ax.fill_between(range(len(inf_hist)), inf_hist, alpha=0.15, color='red')
ax.set_xlabel('Number of Seeds ($k$)', fontsize=11)
ax.set_ylabel('Expected Influence (adoption rate)', fontsize=11)
ax.set_title('Greedy Influence Maximisation - Migration Network\n'
             'Each step adds the department with the highest marginal gain',
             fontsize=12)
ax.grid(alpha=0.3)

# Annotate marginal gains
for i in range(1, len(inf_hist)):
    gain = inf_hist[i] - inf_hist[i-1]
    ax.annotate(f'+{gain:.3f}', (i, inf_hist[i]),
                xytext=(5, 10), textcoords='offset points',
                fontsize=8, color='darkred')

plt.tight_layout()
plt.show()

print(f"\nOptimal seed set: {optimal_seeds}")
```

```
Greedy Seed Selection (  = 1.5, budget k = 5):

  Step 1: selected 'Valle' (marginal gain = 1.0000, total influence = 1.0000)
  Step 2: selected 'Atlántida' (marginal gain = 0.0000, total influence = 1.0000)
  Step 3: selected 'Yoro' (marginal gain = 0.0000, total influence = 1.0000)
  Step 4: selected 'Santa Bárbara' (marginal gain = 0.0000, total influence = 1.0000)
  Step 5: selected 'Choluteca' (marginal gain = 0.0000, total influence = 1.0000)
```

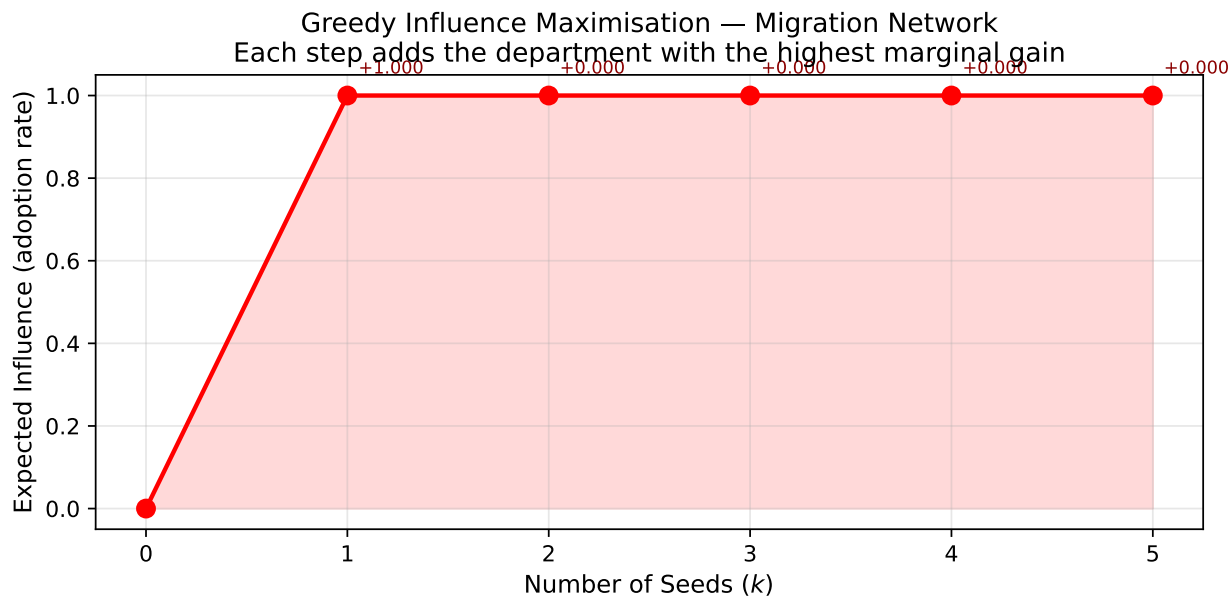


Figure 16: Greedy influence maximisation on the migration network

```
Optimal seed set: {'Santa Bárbara', 'Yoro', 'Atlántida', 'Choluteca', 'Valle'}
```

**How the Greedy Algorithm Works**

The greedy algorithm works as follows:

1. Start with an empty seed set $S = \emptyset$.
2. At each step, evaluate every remaining candidate department: "If I add this department to $S$, how much does the expected cascade size increase?"
3. Select the department with the **highest marginal gain**.
4. Repeat until $|S| = k$ (budget exhausted).

The **submodularity** of influence (diminishing returns: each additional seed helps less) guarantees this greedy approach achieves at least $1 - 1/e \approx 63\%$ of the optimal influence, even though finding the true optimum is NP-hard (Kempe, Kleinberg & Tardos, 2003).
The **marginal gain curve** (annotated numbers on the plot) reveals diminishing returns: the first seed department provides the largest boost, and each subsequent one adds less. The point where marginal gains flatten indicates the "efficient frontier" — beyond which additional seeds provide little extra benefit.

**Policy Implication: Where Should the Government Invest?**

This greedy seeding result provides a **direct, actionable answer** to the question: *"If Honduras has a limited development budget, which departments should receive targeted investment to maximise the retention effect?"*
The optimal seed set represents the departments where local investment has the greatest **multiplier effect** through network cascades:

1. **First priority:** The first department selected by the algorithm is the single most impactful location for development investment — its effects cascade through the migration network to

influence the most other departments.

2. **Diminishing returns inform budgeting:** The marginal gain curve tells policymakers at what point adding more target municipalities stops being cost-effective. If gains flatten after 3 seeds, a 3-department intervention captures most of the benefit of a 5-department one at lower cost.

3. **Network-informed vs. population-based targeting:** The algorithm may select departments that are NOT the largest or most obvious choices — a moderately-sized department that bridges two migration clusters can be more influential than a large but peripheral one. This is the power of the network approach: **optimal intervention targets are determined by connectivity, not just need or population.**

4. **Practical applications:** This methodology can guide placement of free trade zones, educational campuses, healthcare facilities, agricultural modernisation programs, or any place-based development intervention.

---

# 10 8 — Opinion Dynamics on the Migration Network (Unit 6)

Unit 6 introduces the **DeGroot learning model** for opinion dynamics. We apply it to model how migration "opinions" (propensity to migrate) propagate through the inter-department network. The migration flow matrix serves as the influence matrix $T$.

## 10.1 8.1 — DeGroot Model

In the DeGroot model, each department $i$ has an initial "migration opinion" $p_i(0) \in [0, 1]$ representing the propensity to migrate. At each time step:

$$p(t + 1) = T \cdot p(t)$$

where $T$ is the row-stochastic **listening matrix** derived from migration flows. We set $T_{ij} \propto w_{ij}$ (flow from $j$ to $i$) with self-weight included.

Intuitively, each department updates its migration propensity by taking a **weighted average** of its own current propensity and those of the departments it receives migrants from. Over time, all departments converge to a single **consensus opinion** — the network-weighted average of initial propensities.

```
#   Build the listening matrix T from migration flows
dept_list = sorted(dept_names.values())
n_dept = len(dept_list)
dept_idx = {d: i for i, d in enumerate(dept_list)}

# Adjacency matrix (with self-loops for "listening to self")
W = np.zeros((n_dept, n_dept))
for u, v, data in G_dept.edges(data=True):
    W[dept_idx[v], dept_idx[u]] = data['weight']  # T_ij = flow from j to i

# Add self-weight: proportional to in-city population
in_city_totals = df[df['in_city'] == 1].groupby('codDepto')['N'].sum()
for code, name in dept_names.items():
```

```
    if code in in_city_totals.index:
        W[dept_idx[name], dept_idx[name]] = in_city_totals[code]

# Row-normalise to get T
row_sums = W.sum(axis=1, keepdims=True)
row_sums[row_sums == 0] = 1  # avoid division by zero
T = W / row_sums

#   Initial opinions: migration rate per department
# Use the actual observed migration rate as initial opinion
dept_mig_rates = df[df['in_city'] == 0].groupby('codDeptoNac')['N'].sum()
dept_total = df.groupby('codDeptoNac')['N'].sum()
mig_rate_init = (dept_mig_rates / dept_total).reindex(range(1, 19), fill_value=0)

# Order p0 alphabetically to match dept_list / T matrix ordering
name_to_code = {v: k for k, v in dept_names.items()}
p0 = np.array([mig_rate_init.get(name_to_code[d], 0) for d in dept_list])

#   Run DeGroot dynamics
T_steps = 50
p_history = np.zeros((T_steps + 1, n_dept))
p_history[0] = p0

for t in range(T_steps):
    p_history[t + 1] = T @ p_history[t]

#   Plot convergence
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

cmap_lines = plt.cm.tab20
for i, dept in enumerate(dept_list):
    ax1.plot(range(T_steps + 1), p_history[:, i],
             color=cmap_lines(i / n_dept), label=dept, linewidth=1.5)

ax1.set_xlabel('Time step $t$')
ax1.set_ylabel('Migration opinion $p_i(t)$')
ax1.set_title('DeGroot Opinion Dynamics\n(Initial opinions = observed migration rates)')
ax1.legend(fontsize=5, ncol=2, loc='center right')
ax1.grid(alpha=0.3)

# Convergence metric
consensus_val = p_history[-1].mean()
deviations = [np.sqrt(np.mean((p_history[t] - consensus_val)**2))
              for t in range(T_steps + 1)]
ax2.plot(deviations, 'k-', linewidth=1.5)
ax2.set_xlabel('Time step $t$')
ax2.set_ylabel('RMSD from consensus')
ax2.set_title('Convergence to Consensus')
ax2.grid(alpha=0.3)
ax2.set_yscale('log')

plt.tight_layout()
plt.show()
```

```
print(f"Consensus migration opinion: {consensus_val:.4f}")
print(f"Initial range: [{p0.min():.4f}, {p0.max():.4f}]")
print(f"Final range: [{p_history[-1].min():.4f}, {p_history[-1].max():.4f}]")
```

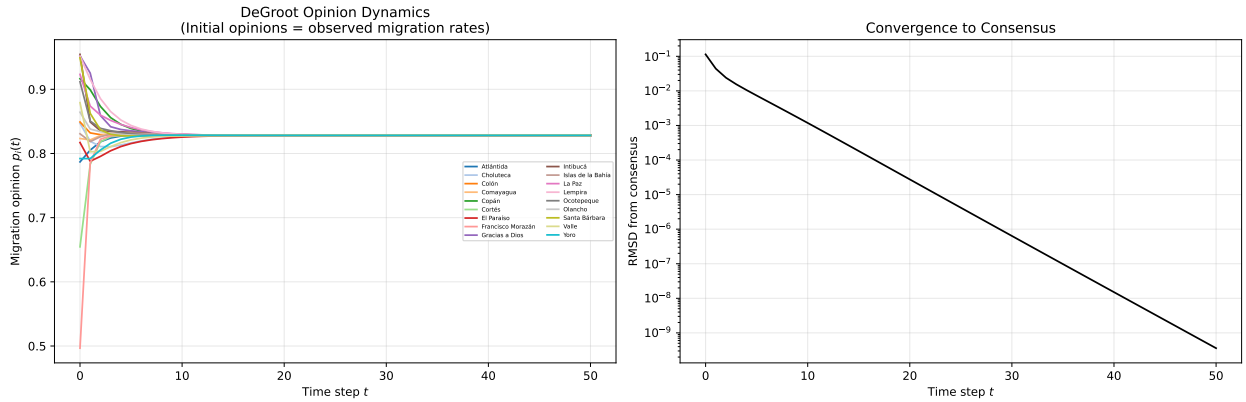


Figure 17: DeGroot opinion dynamics on the migration network

```
Consensus migration opinion: 0.8282
Initial range: [0.4966, 0.9544]
Final range: [0.8282, 0.8282]
```

**Interpreting the DeGroot Convergence**

**Left panel (Opinion Trajectories):** Each coloured line represents one department's migration propensity over time. The key observations are:

- **Initial spread:** The vertical spread at $t = 0$ reflects current heterogeneity in migration rates across departments. Departments with high initial propensity are major migrant senders; those with low propensity have more sedentary populations.
- **Convergence:** As $t$ increases, all lines converge toward a **single consensus value** $p(\infty)$. This is the defining feature of the DeGroot model: social influence through the migration network eventually homogenises opinions.
- **Speed of convergence:** How quickly the lines bunch together depends on the network's spectral properties (see Section 8.2).

**Right panel (RMSD):** The root-mean-square deviation from consensus drops exponentially — linear on the log scale. The slope is determined by $|\lambda_2|$, the second-largest eigenvalue of $T$.
The **consensus value** itself represents the long-run "national migration norm" that would emerge if departments continuously influenced each other through their migration connections.

```
#   Heatmap of the T matrix
fig, ax = plt.subplots(figsize=(10, 8))
im = ax.imshow(T, cmap='YlOrRd', aspect='auto', vmin=0)
ax.set_xticks(range(n_dept))
ax.set_yticks(range(n_dept))
ax.set_xticklabels(dept_list, rotation=90, fontsize=7)
ax.set_yticklabels(dept_list, fontsize=7)
ax.set_xlabel('Influencer (column $j$ = department being listened to)', fontsize=10)
```

```python
ax.set_ylabel('Listener (row $i$ = department being influenced)', fontsize=10)
ax.set_title('Listening Matrix $T$\n'
             '$T_{ij}$ = weight that department $i$ places on department $j$\'s opinion\n'
             'Diagonal = self-listening (staying); off-diagonal = migration influence',
             fontsize=11)
plt.colorbar(im, ax=ax, label='$T_{ij}$ (row-stochastic weight)')
plt.tight_layout()
plt.show()

# Report diagonal dominance
diag_vals = np.diag(T)
print("Self-listening weights (diagonal of T):")
for i, dept in enumerate(dept_list):
    print(f"  {dept:25s}  T_ii = {diag_vals[i]:.4f}  "
          f"({'mostly listens to self' if diag_vals[i] > 0.7 else 'influenced by others'})")
```

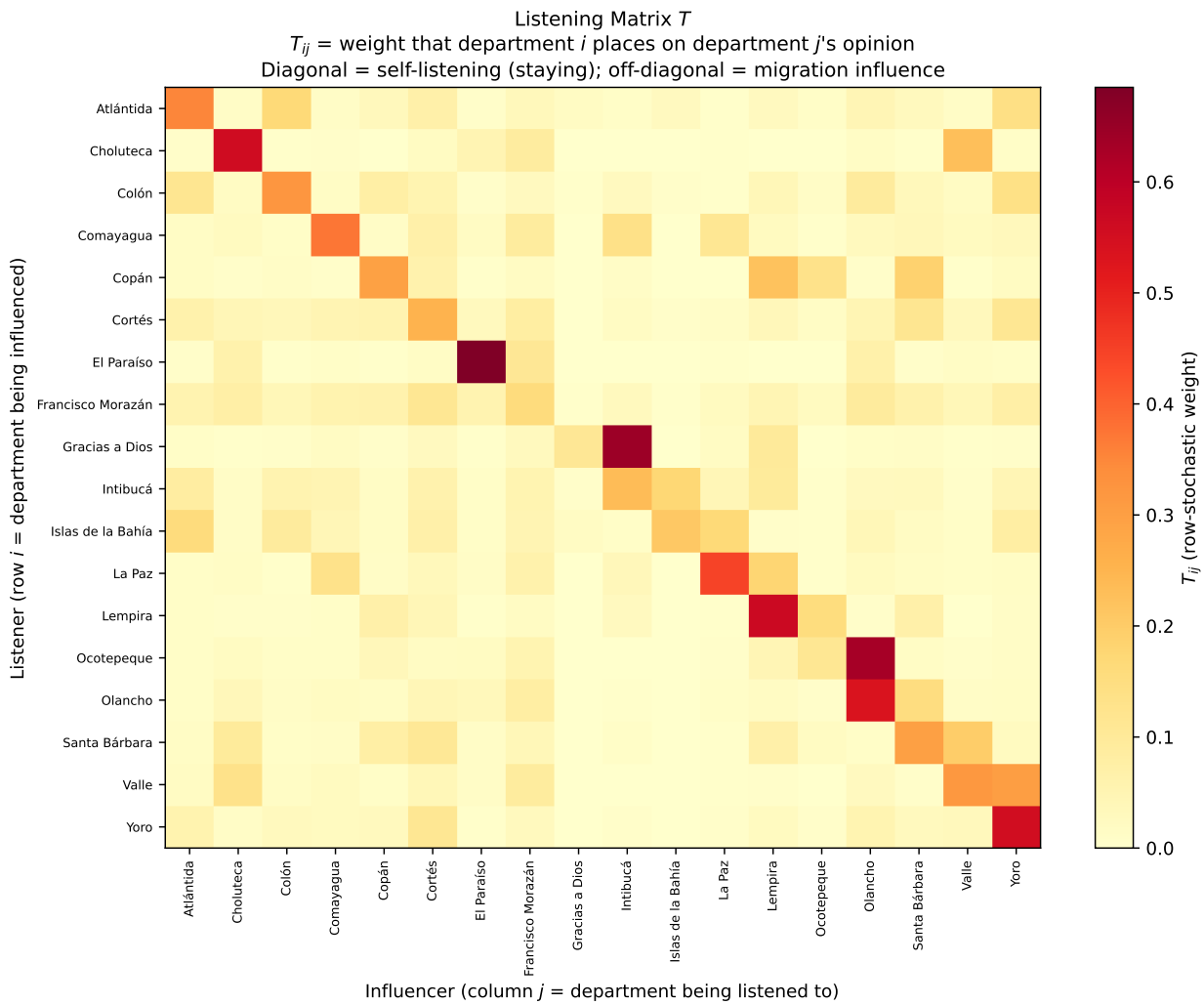


Figure 18: The listening matrix T — who influences whom in the migration network?

```
Self-listening weights (diagonal of T):
```

```
Atlántida                  T_ii = 0.3516  (influenced by others)
Choluteca                  T_ii = 0.5569  (influenced by others)
Colón                      T_ii = 0.3189  (influenced by others)
Comayagua                  T_ii = 0.3742  (influenced by others)
Copán                      T_ii = 0.2969  (influenced by others)
Cortés                     T_ii = 0.2516  (influenced by others)
El Paraíso                 T_ii = 0.6851  (influenced by others)
Francisco Morazán          T_ii = 0.1569  (influenced by others)
Gracias a Dios             T_ii = 0.1081  (influenced by others)
Intibucá                   T_ii = 0.2338  (influenced by others)
Islas de la Bahía          T_ii = 0.2082  (influenced by others)
La Paz                     T_ii = 0.4461  (influenced by others)
Lempira                    T_ii = 0.5673  (influenced by others)
Ocotepeque                 T_ii = 0.1103  (influenced by others)
Olancho                    T_ii = 0.5331  (influenced by others)
Santa Bárbara              T_ii = 0.2989  (influenced by others)
Valle                      T_ii = 0.3160  (influenced by others)
Yoro                       T_ii = 0.5559  (influenced by others)
```

**Reading the Listening Matrix Heatmap**

The $T$ matrix reveals **who listens to whom** in the migration network:

- **Diagonal cells** ($T_{ii}$): The self-listening weight. A high diagonal means the department mostly "listens to itself" — its migration opinion is resistant to outside influence. This corresponds to departments where most people stay put (high in-city ratio).
- **Off-diagonal cells** ($T_{ij}$): The influence of department $j$ on department $i$. Bright off-diagonal cells indicate strong influence channels. A department with a bright *row* is highly susceptible to outside influence; a department with a bright *column* is an influential opinion leader.
- **The dominant columns** (e.g., Francisco Morazán, Cortés) are the departments whose migration patterns most strongly shape others' opinions.

## 10.2 8.2 — Influence Vector and Spectral Analysis

The consensus opinion is $p(\infty) = \mathbf{v}' p(0)$, where $\mathbf{v}$ is the left eigenvector of $T$ for eigenvalue 1 (the **influence vector**). The **speed of convergence** is governed by $|\lambda_2|$, the second-largest eigenvalue of $T$ (Unit 6).

```
#   Compute eigenvalues of T
eigenvalues = np.linalg.eigvals(T)
eigenvalues_sorted = sorted(eigenvalues, key=lambda x: -abs(x))

# Influence vector: left eigenvector for  =1
T_sparse = sp.csr_matrix(T.T)
vals, vecs = eigs(T_sparse, k=2, which='LM')
# The eigenvector for     1
idx_lam1 = np.argmax(np.abs(vals))
v = np.abs(vecs[:, idx_lam1].real)
v = v / v.sum()  # normalise

lam2 = abs(eigenvalues_sorted[1])

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
```

```
# Eigenvalue spectrum
ax1.scatter(range(len(eigenvalues_sorted)), [abs(e) for e in eigenvalues_sorted],
            c='steelblue', s=50)
ax1.axhline(1, color='red', ls='--', alpha=0.5, label='$\\lambda_1 = 1$')
ax1.axhline(lam2, color='orange', ls='--', alpha=0.5,
            label=f'$|\\lambda_2| = {lam2:.4f}$')
ax1.set_xlabel('Eigenvalue index')
ax1.set_ylabel('$|\\lambda|$')
ax1.set_title('Eigenvalue Spectrum of $T$')
ax1.legend()
ax1.grid(alpha=0.3)

# Influence vector
sorted_idx = np.argsort(v)[::-1]
ax2.barh(range(n_dept), v[sorted_idx], color='coral')
ax2.set_yticks(range(n_dept))
ax2.set_yticklabels([dept_list[i] for i in sorted_idx], fontsize=7)
ax2.set_xlabel('Influence $v_i$')
ax2.set_title('Influence Vector (DeGroot consensus weights)')
ax2.invert_yaxis()

plt.tight_layout()
plt.show()

print(f"| | = {lam2:.4f}")
print(f"Spectral gap (1 - | |) = {1 - lam2:.4f}")
print(f"Estimated convergence time  log(1/ ) / log(1/| |): "
      f"t*  {np.log(10) / np.log(1/lam2):.0f} steps (for  =0.1)")
```

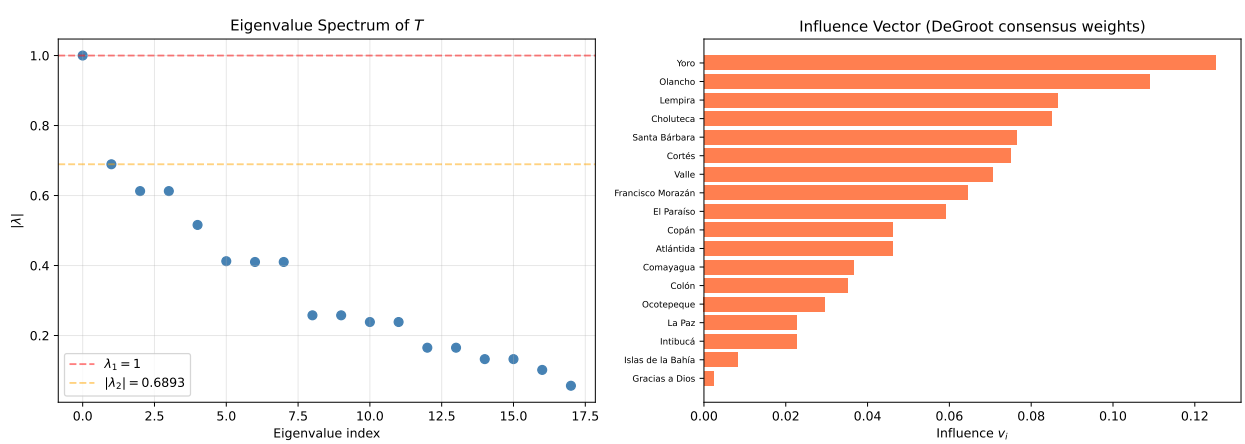


Figure 19: Influence vector and eigenvalue spectrum of the migration listening matrix

```
| | = 0.6893
Spectral gap (1 - | |) = 0.3107
Estimated convergence time  log(1/ ) / log(1/| |): t*  6 steps (for  =0.1)
```

## 10.3 8.3 — Friedkin-Johnsen Model

The **Friedkin-Johnsen** (FJ) model from Unit 6 extends DeGroot by allowing each department to retain a fraction of its **innate opinion** (observed migration rate). The susceptibility parameter $\alpha_i$ captures how much department $i$ listens to the network vs. its own experience:

$$p_i(t+1) = \alpha_i \sum_j T_{ij} p_j(t) + (1-\alpha_i) s_i$$

Steady state: $p^* = (I - \Lambda T)^{-1}(I - \Lambda)s$

```
#   Susceptibility: proportional to connectivity
# Departments more connected to the migration network are more susceptible
degrees = np.array([G_dept.degree(d, weight='weight') for d in dept_list])
alpha = 0.3 + 0.6 * (degrees / degrees.max())  # range [0.3, 0.9]

#   Solve FJ steady state
I = np.eye(n_dept)
Lambda = np.diag(alpha)
s = p0  # innate opinions = observed migration rates

# p* = (I - Λ T)^{-1} (I - Λ) s
lhs = I - Lambda @ T
rhs = (I - Lambda) @ s
p_star_fj = np.linalg.solve(lhs, rhs)

#   Compare DeGroot vs FJ
fig, ax = plt.subplots(figsize=(10, 6))

sorted_idx = np.argsort(p0)[::-1]
x = np.arange(n_dept)

ax.bar(x - 0.2, p0[sorted_idx], 0.2, label='Initial (observed)', color='lightblue')
ax.bar(x, p_history[-1][sorted_idx], 0.2, label='DeGroot consensus', color='coral')
ax.bar(x + 0.2, p_star_fj[sorted_idx], 0.2, label='Friedkin-Johnsen', color='green')

ax.set_xticks(x)
ax.set_xticklabels([dept_list[i] for i in sorted_idx], rotation=45,
                   ha='right', fontsize=7)
ax.set_ylabel('Migration Propensity')
ax.set_title('Migration Opinions: Initial vs DeGroot vs Friedkin-Johnsen')
ax.legend()
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

print("Friedkin-Johnsen steady-state opinions:")
for i in sorted_idx:
    print(f"  {dept_list[i]:25s}  initial={p0[i]:.4f}  "
          f"DeGroot={p_history[-1, i]:.4f}  FJ={p_star_fj[i]:.4f}")
```

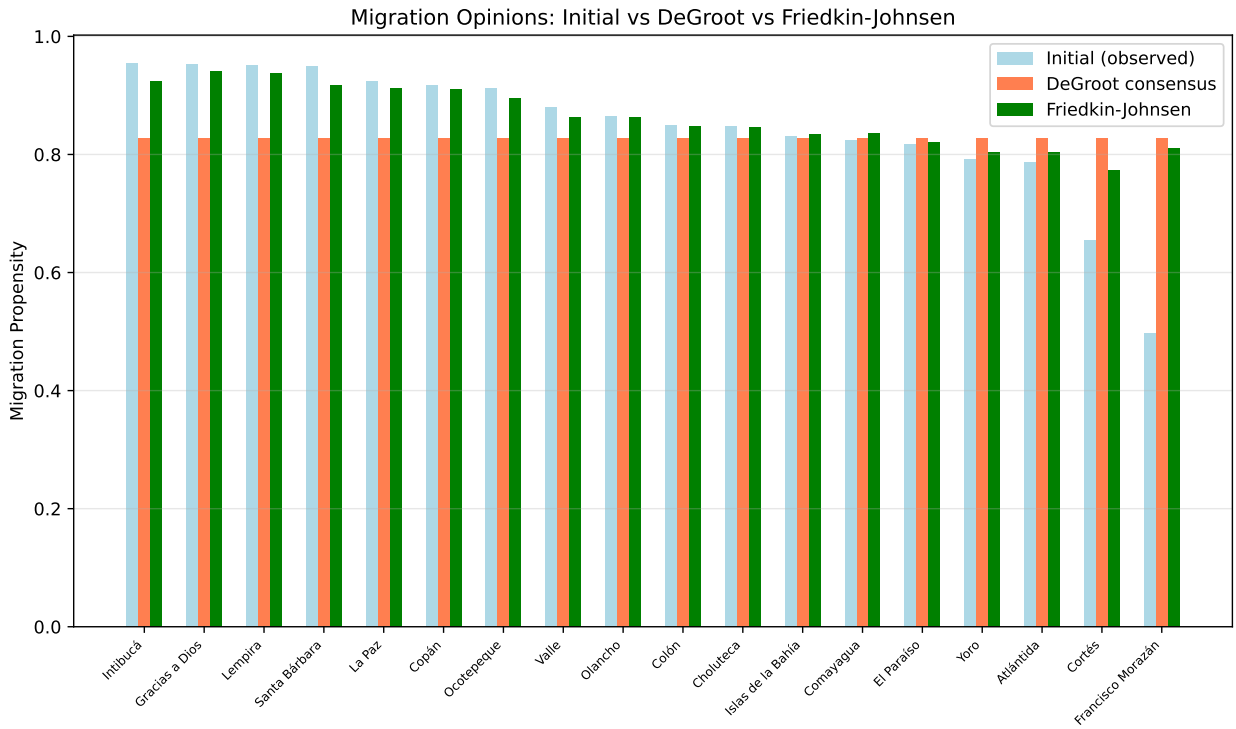


Figure 20: Friedkin-Johnsen steady state vs. DeGroot consensus

```
Friedkin-Johnsen steady-state opinions:
  Intibucá                  initial=0.9544  DeGroot=0.8282  FJ=0.9242
  Gracias a Dios            initial=0.9522  DeGroot=0.8282  FJ=0.9406
  Lempira                   initial=0.9518  DeGroot=0.8282  FJ=0.9375
  Santa Bárbara             initial=0.9492  DeGroot=0.8282  FJ=0.9177
  La Paz                    initial=0.9233  DeGroot=0.8282  FJ=0.9122
  Copán                     initial=0.9165  DeGroot=0.8282  FJ=0.9098
  Ocotepeque                initial=0.9117  DeGroot=0.8282  FJ=0.8957
  Valle                     initial=0.8792  DeGroot=0.8282  FJ=0.8625
  Olancho                   initial=0.8643  DeGroot=0.8282  FJ=0.8635
  Colón                     initial=0.8491  DeGroot=0.8282  FJ=0.8482
  Choluteca                 initial=0.8471  DeGroot=0.8282  FJ=0.8460
  Islas de la Bahía         initial=0.8307  DeGroot=0.8282  FJ=0.8349
  Comayagua                 initial=0.8233  DeGroot=0.8282  FJ=0.8361
  El Paraíso                initial=0.8171  DeGroot=0.8282  FJ=0.8201
  Yoro                      initial=0.7921  DeGroot=0.8282  FJ=0.8041
  Atlántida                 initial=0.7869  DeGroot=0.8282  FJ=0.8045
  Cortés                    initial=0.6546  DeGroot=0.8282  FJ=0.7727
  Francisco Morazán         initial=0.4966  DeGroot=0.8282  FJ=0.8100
```

**Interpreting the DeGroot vs Friedkin-Johnsen Comparison**

The grouped bar chart reveals a fundamental difference between two models of social influence:

- **Blue bars (Initial):** The observed migration propensity of each department. The variation reflects real economic and social differences across Honduras.
- **Orange bars (DeGroot):** Complete convergence to a single national consensus. All depart-

ments end up with the *same* migration propensity, regardless of their initial conditions. This is unrealistic but instructive: it shows what would happen if migration norms spread without resistance.

- **Green bars (Friedkin-Johnsen):** Partial convergence with persistent heterogeneity. Each department's steady state is a compromise between the network consensus pull and its own innate conditions. More connected departments (higher $\alpha_i$) are pulled further toward consensus; isolated departments (lower $\alpha_i$) stay closer to their initial values.

The FJ model captures a crucial real-world phenomenon: **migration patterns in isolated departments like Gracias a Dios or Islas de la Bahía will not converge to the national average**, no matter how strong migration trends are elsewhere. Their structural disconnection from the migration network creates natural resistance to homogenising social forces.

❗ Policy Implication: Persistent Disagreement and Tailored Policy

The Friedkin-Johnsen results have profound implications for migration policy design:

1. **One-size-fits-all national policies will fail.** The FJ model demonstrates that structurally isolated departments will not respond to national migration trends. A "national employment program" will have vastly different effects in well-connected Cortés (which converges toward the national norm) versus isolated Gracias a Dios (which maintains its own equilibrium).

2. **Network rewiring.** To bring isolated departments into the national economic conversation, policymakers must **add connections** — transport links, communication infrastructure, economic partnerships — that increase $\alpha_i$ and pull these departments toward the national mainstream.

3. **Preserving beneficial isolation.** Conversely, some departments with low out-migration may benefit from their isolation. If a department has a healthy local economy and retains its population, forcing it into the migration mainstream (by building a highway to the capital) could paradoxically *increase* out-migration as it gets exposed to urban pull factors. The FJ model helps identify which departments should be **connected** and which should be left in productive isolation.

4. **Susceptibility mapping.** The $\alpha_i$ values provide a direct "policy receptivity" score: departments with high $\alpha_i$ are more responsive to network-level changes (information campaigns, peer effects), while those with low $\alpha_i$ require direct, local interventions.

---

# 11 9 — Routing Analogy: Migration as a Flow Network (Unit 4)

Unit 4 studies **routing on transportation networks** with congestion effects. We draw an analogy: migration flows create "congestion" in destination cities (housing costs, labour market competition), so the Nash equilibrium of migration may be socially inefficient — a migration analogue of **Braess's paradox**.

## 11.1 9.1 — Migration Flow Game Setup

We model each department as choosing between staying (self-loop) or migrating to Tegucigalpa or San Pedro Sula. The "latency" of each destination increases with the number of migrants (congestion).

```
#   Simple 2-destination flow game
# For each origin department, model the choice:
#   Stay (cost = c_stay)  vs  Migrate to Tegu (cost = c_tegu(x))  vs  SPS (cost = c_sps(x))
#
# Congestion: l(x) = base_cost +   * (x / capacity)
# where x = total flow to that destination

# Parameters
beta = 0.5  # congestion coefficient

# Get actual flows
actual_to_tegu = df[df['capital'] == 1].groupby('codDeptoNac')['N'].sum()
actual_to_sps = df[df['sps'] == 1].groupby('codDeptoNac')['N'].sum()
actual_stay = df[df['in_city'] == 1].groupby('codDeptoNac')['N'].sum()

total_to_tegu = actual_to_tegu.sum()
total_to_sps = actual_to_sps.sum()
total_stay = actual_stay.sum()

# Capacity (proportional to city population)
cap_tegu = 1200000  # Tegucigalpa metro population
cap_sps = 800000    # SPS metro population

# Latency at current flows
latency_tegu = 1 + beta * (total_to_tegu / cap_tegu)
latency_sps = 1 + beta * (total_to_sps / cap_sps)
latency_stay = 0.5  # base cost of staying (lower)

# Total cost = Σ x_i * l_i(x_i)
total_cost_eq = (total_to_tegu * latency_tegu +
                 total_to_sps * latency_sps +
                 total_stay * latency_stay)

# Social optimum: redistribute to equalise marginal costs
# l'(x) =  /cap, marginal cost = l(x) + x * l'(x) = 1 + 2 (x/cap)
# At optimum: marginal costs should be equal across used routes

print("=== Migration Flow Game (Routing Analogy) ===\n")
print(f"Current flows:")
print(f"  To Tegucigalpa:  {total_to_tegu:>10,.0f}  "
      f"(latency = {latency_tegu:.3f})")
print(f"  To San Pedro Sula: {total_to_sps:>10,.0f}  "
      f"(latency = {latency_sps:.3f})")
print(f"  Staying:          {total_stay:>10,.0f}  "
      f"(latency = {latency_stay:.3f})")
print(f"\nTotal equilibrium cost: {total_cost_eq:,.0f}")
print(f"\nMarginal cost to Tegu: {1 + 2*beta*(total_to_tegu/cap_tegu):.3f}")
print(f"Marginal cost to SPS:  {1 + 2*beta*(total_to_sps/cap_sps):.3f}")

# Visualise
fig, ax = plt.subplots(figsize=(8, 5))
categories = ['Tegucigalpa', 'San Pedro Sula', 'Staying']
flows = [total_to_tegu, total_to_sps, total_stay]
```

```python
latencies = [latency_tegu, latency_sps, latency_stay]
colors = ['#e74c3c', '#3498db', '#2ecc71']

bars = ax.bar(categories, flows, color=colors, alpha=0.8)
ax.set_ylabel('Number of Migrants')
ax.set_title('Migration Flows and Congestion Costs')

# Add latency annotation
for bar, lat in zip(bars, latencies):
    ax.annotate(f'Latency = {lat:.3f}',
                xy=(bar.get_x() + bar.get_width()/2, bar.get_height()),
                xytext=(0, 5), textcoords='offset points',
                ha='center', fontsize=9)

plt.tight_layout()
plt.show()
```

```
=== Migration Flow Game (Routing Analogy) ===

Current flows:
  To Tegucigalpa:   7,137,000  (latency = 3.974)
  To San Pedro Sula:  2,904,090  (latency = 2.815)
  Staying:           4,382,483  (latency = 0.500)

Total equilibrium cost: 38,727,072

Marginal cost to Tegu: 6.947
Marginal cost to SPS:  4.630
```

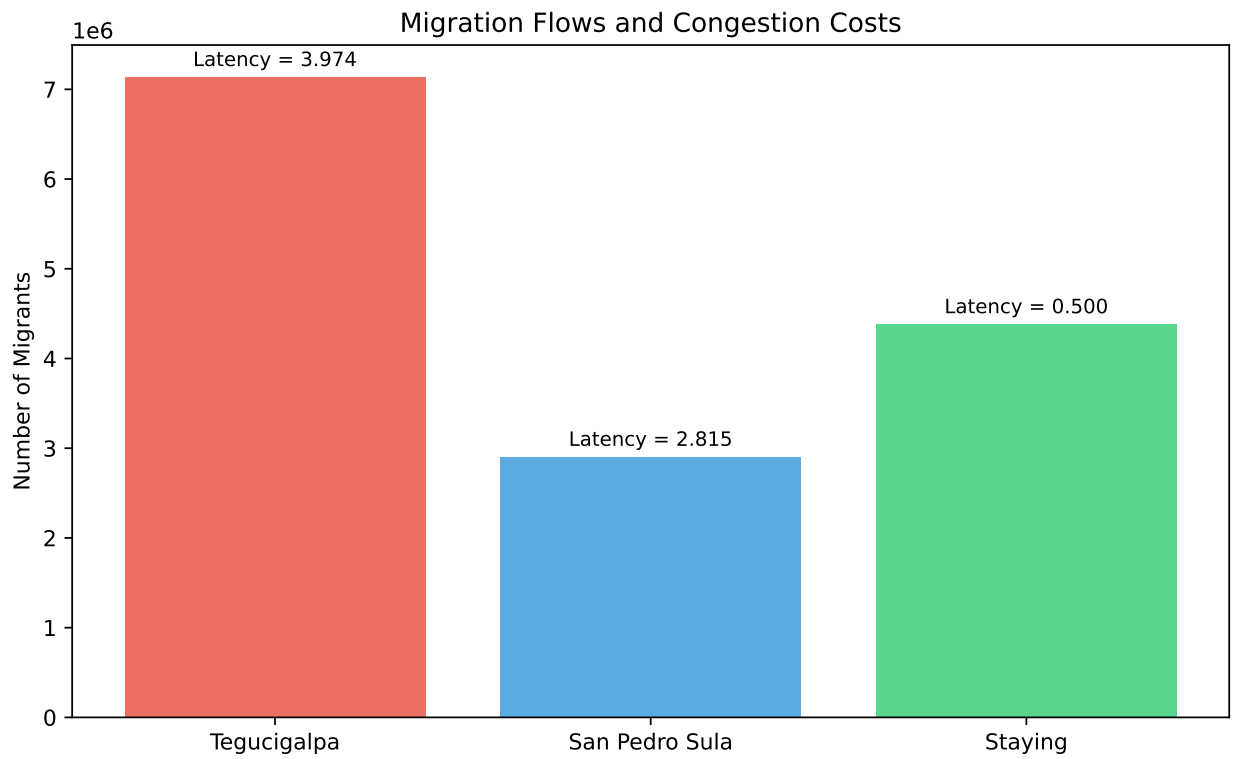


Figure 21: Migration equilibrium vs social optimum (routing analogy)

```
#   Congestion cost curves
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

x_range = np.linspace(0, max(total_to_tegu, total_to_sps) * 1.5, 500)

# Latency curves
for cap, city, color, actual_flow in [
    (cap_tegu, 'Tegucigalpa', '#e74c3c', total_to_tegu),
    (cap_sps, 'San Pedro Sula', '#3498db', total_to_sps)
]:
    latency_curve = 1 + beta * (x_range / cap)
    marginal_curve = 1 + 2 * beta * (x_range / cap)
    ax1.plot(x_range, latency_curve, color=color, linewidth=2, label=f'{city} latency')
    ax1.plot(x_range, marginal_curve, color=color, linewidth=2, ls='--',
             label=f'{city} marginal cost')
    ax1.axvline(actual_flow, color=color, alpha=0.3, ls=':')
    ax1.scatter([actual_flow], [1 + beta * actual_flow / cap],
                color=color, s=100, zorder=5, edgecolors='black')

ax1.set_xlabel('Number of migrants to city', fontsize=10)
ax1.set_ylabel('Cost (latency / congestion)', fontsize=10)
ax1.set_title('Congestion Cost Curves\n'
              'Solid = latency $\ell(x)$; Dashed = marginal cost $\ell(x) + x\ell\'(x)$',
              fontsize=11)
ax1.legend(fontsize=8)
ax1.grid(alpha=0.3)
```

```
# Price of Anarchy visualisation
# Compute social optimum (equalise marginal costs)
# marginal cost Tegu = 1 + 2*beta*x_T/cap_T
# marginal cost SPS  = 1 + 2*beta*x_S/cap_S
# total migrants to cities = total_to_tegu + total_to_sps = T_total
T_total = total_to_tegu + total_to_sps
if T_total > 0:
    # At social optimum: 1 + 2*beta*x_T*/cap_T = 1 + 2*beta*x_S*/cap_S
    # => x_T*/cap_T = x_S*/cap_S => x_T* = (cap_T/(cap_T+cap_S)) * T_total
    x_T_opt = (cap_tegu / (cap_tegu + cap_sps)) * T_total
    x_S_opt = T_total - x_T_opt
    lat_T_opt = 1 + beta * (x_T_opt / cap_tegu)
    lat_S_opt = 1 + beta * (x_S_opt / cap_sps)
    total_cost_opt = x_T_opt * lat_T_opt + x_S_opt * lat_S_opt + total_stay * latency_stay
    total_cost_nash = total_cost_eq
    poa = total_cost_nash / total_cost_opt if total_cost_opt > 0 else 1

    ax2.bar(['Nash Equilibrium\n(current)'], [total_cost_nash],
            color='#e74c3c', alpha=0.7, edgecolor='black')
    ax2.bar(['Social Optimum\n(redistributed)'], [total_cost_opt],
            color='#2ecc71', alpha=0.7, edgecolor='black')
    ax2.set_ylabel('Total Social Cost', fontsize=10)
    ax2.set_title(f'Price of Anarchy = {poa:.3f}\n'
                  f'(Inefficiency of selfish migration choices)',
                  fontsize=11)
    ax2.grid(axis='y', alpha=0.3)

    # Annotate the gap
    waste = total_cost_nash - total_cost_opt
    ax2.annotate(f'Waste = {waste:,.0f}\n({(poa-1)*100:.1f}% excess cost)',
                 xy=(0.5, (total_cost_nash + total_cost_opt) / 2),
                 fontsize=10, ha='center', color='darkred',
                 fontweight='bold')
else:
    ax2.text(0.5, 0.5, 'No city migration data available',
             transform=ax2.transAxes, ha='center')

plt.tight_layout()
plt.show()

if T_total > 0:
    print(f"\n=== Price of Anarchy Analysis ===")
    print(f"  Nash equilibrium cost:  {total_cost_nash:>12,.0f}")
    print(f"  Social optimum cost:    {total_cost_opt:>12,.0f}")
    print(f"  Price of Anarchy:       {poa:.4f}")
    print(f"  Wasted resources:       {waste:>12,.0f} ({(poa-1)*100:.1f}%)")
    print(f"\n  Optimal reallocation:")
    print(f"    Tegucigalpa: {total_to_tegu:,.0f} \u2192 {x_T_opt:,.0f} "
          f"(change: {x_T_opt - total_to_tegu:+,.0f})")
    print(f"    SPS:         {total_to_sps:,.0f} \u2192 {x_S_opt:,.0f} "
          f"(change: {x_S_opt - total_to_sps:+,.0f})")
```

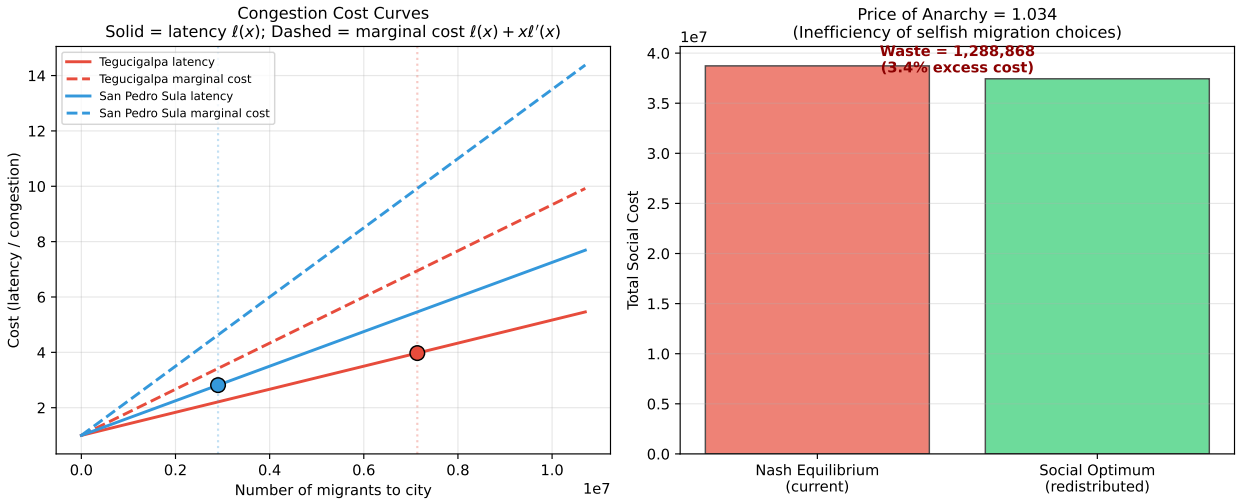


Figure 22: Congestion cost curves: How latency increases with migration flow

```
=== Price of Anarchy Analysis ===
  Nash equilibrium cost:    38,727,072
  Social optimum cost:      37,438,204
  Price of Anarchy:       1.0344
  Wasted resources:          1,288,868 (3.4%)

  Optimal reallocation:
    Tegucigalpa: 7,137,000 → 6,024,654 (change: -1,112,346)
    SPS:         2,904,090 → 4,016,436 (change: +1,112,346)
```

**Understanding the Routing Analogy**

The routing analogy applies game theory from Unit 4 to migration:

- **Each migrant is a selfish agent** choosing a destination to minimise their personal "cost" (difficulty of settling: housing prices, job competition, congestion). This is analogous to drivers choosing routes in a traffic network.
- **Congestion externality:** Each additional migrant to a city increases costs for *everyone already there* (housing prices rise, jobs become scarcer). But the individual migrant only considers their own cost, not the externality they impose on others.
- **Nash equilibrium:** The observed migration pattern is a Nash equilibrium where no individual can unilaterally improve by switching destinations.
- **Social optimum:** A central planner considering total social cost would redistribute migrants more evenly, reducing congestion in the most overcrowded city.
- **Price of Anarchy (PoA):** The ratio of Nash cost to social optimum cost. PoA = 1.00 means the equilibrium is efficient; PoA > 1 quantifies the inefficiency of selfish migration choices.

**The congestion cost curves** (left panel) show how costs increase with flows. The gap between the latency curve (individual cost) and marginal cost curve (social cost) is the externality that migrants fail to internalise.

**!** Policy Implication: Braess's Paradox and Urban Planning

The routing analogy reveals that **decentralised migration decisions lead to socially inefficient outcomes**:

1. **Congestion pricing analogy.** Just as cities use congestion charges for traffic, governments could implement "de-concentration incentives" — tax breaks, subsidised housing, or guaranteed public employment in secondary cities — to redirect migration from congested Tegucigalpa and SPS toward underutilised secondary urban centres.

2. **Braess's Paradox risk.** Adding a new highway between two migration clusters could paradoxically *increase* total congestion by enabling even more people to access already-crowded destinations. Infrastructure investments should be evaluated not just for their direct effects but for how they reshape migration equilibria.

3. **The social cost gap** (waste from Nash vs. optimum) quantifies the **economic cost of laissez-faire migration policy.** It provides a dollar-denominated argument for active regional development policy: the government should invest *up to* this amount in migration-redirecting programs, as any lesser investment pays for itself.

4. **Secondary city development** as a corrective mechanism: if the social optimum redistributes migrants toward SPS or other cities, the policy prescription is to invest in those destinations' capacity (housing, jobs, infrastructure) to absorb the redirected flow.

---

# 12 10 — Summary of Network Analysis Findings

```
#   Summary dashboard
fig, axes = plt.subplots(2, 3, figsize=(14, 10))

# 1. Network size
ax = axes[0, 0]
metrics = ['Dept Nodes', 'Dept Edges', 'Mun Nodes', 'Mun Edges']
values = [G_dept.number_of_nodes(), G_dept.number_of_edges(),
          G_mun_connected.number_of_nodes(), G_mun_connected.number_of_edges()]
colors_sm = ['#3498db', '#3498db', '#e74c3c', '#e74c3c']
ax.barh(metrics, values, color=colors_sm, alpha=0.8)
ax.set_title('Network Size', fontsize=11, fontweight='bold')
for i, v in enumerate(values):
    ax.text(v, i, f' {v:,}', va='center', fontsize=9)
ax.set_xscale('log')

# 2. Clustering comparison
ax = axes[0, 1]
clust_labels = ['Observed', 'ER Null', 'Config Model']
clust_vals = [obs_clustering, np.mean(er_clustering), np.mean(cm_clustering)]
clust_colors = ['#e74c3c', '#3498db', '#2ecc71']
ax.bar(clust_labels, clust_vals, color=clust_colors, alpha=0.8,
       edgecolor='black')
ax.set_title('Clustering Coefficient', fontsize=11, fontweight='bold')
```

```python
ax.set_ylabel('Transitivity')
for i, v in enumerate(clust_vals):
    ax.text(i, v, f'{v:.4f}', ha='center', va='bottom', fontsize=9)

# 3. Modularity
ax = axes[0, 2]
mod_labels = ['Department\nNetwork', 'Municipality\nNetwork']
mod_vals = [Q, Q_mun]
ax.bar(mod_labels, mod_vals, color=['#9b59b6', '#f39c12'],
       alpha=0.8, edgecolor='black')
ax.set_title('Community Modularity', fontsize=11, fontweight='bold')
ax.set_ylabel('Q')
ax.axhline(0.3, color='red', ls='--', alpha=0.5, label='Threshold (Q=0.3)')
ax.legend(fontsize=8)
for i, v in enumerate(mod_vals):
    ax.text(i, v, f'{v:.3f}', ha='center', va='bottom', fontsize=9)

# 4. Top 5 departments by PageRank
ax = axes[1, 0]
top5_pr = centrality_df.nlargest(5, 'PageRank')
ax.barh(top5_pr['Department'], top5_pr['PageRank'],
        color='#e74c3c', alpha=0.8)
ax.set_title('Top 5 Depts (PageRank)', fontsize=11, fontweight='bold')
ax.invert_yaxis()

# 5. Spectral gap
ax = axes[1, 1]
ax.bar(['| |', 'Spectral Gap\n(1 - | |)'],
       [lam2, 1 - lam2],
       color=['#e74c3c', '#2ecc71'], alpha=0.8, edgecolor='black')
ax.set_title('DeGroot Convergence Speed', fontsize=11, fontweight='bold')
for i, v in enumerate([lam2, 1 - lam2]):
    ax.text(i, v, f'{v:.4f}', ha='center', va='bottom', fontsize=9)

# 6. Price of Anarchy
ax = axes[1, 2]
if T_total > 0:
    ax.bar(['PoA'], [poa], color='#e74c3c', alpha=0.8, edgecolor='black')
    ax.axhline(1, color='green', ls='--', label='Efficient (PoA=1)')
    ax.set_title('Price of Anarchy\n(Migration Routing)', fontsize=11,
                 fontweight='bold')
    ax.legend(fontsize=8)
    ax.text(0, poa, f'{poa:.3f}', ha='center', va='bottom', fontsize=11,
            fontweight='bold')
else:
    ax.text(0.5, 0.5, 'N/A', transform=ax.transAxes, ha='center', fontsize=14)

plt.suptitle('Network Analysis of Honduras Internal Migration - Key Metrics',
             fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```

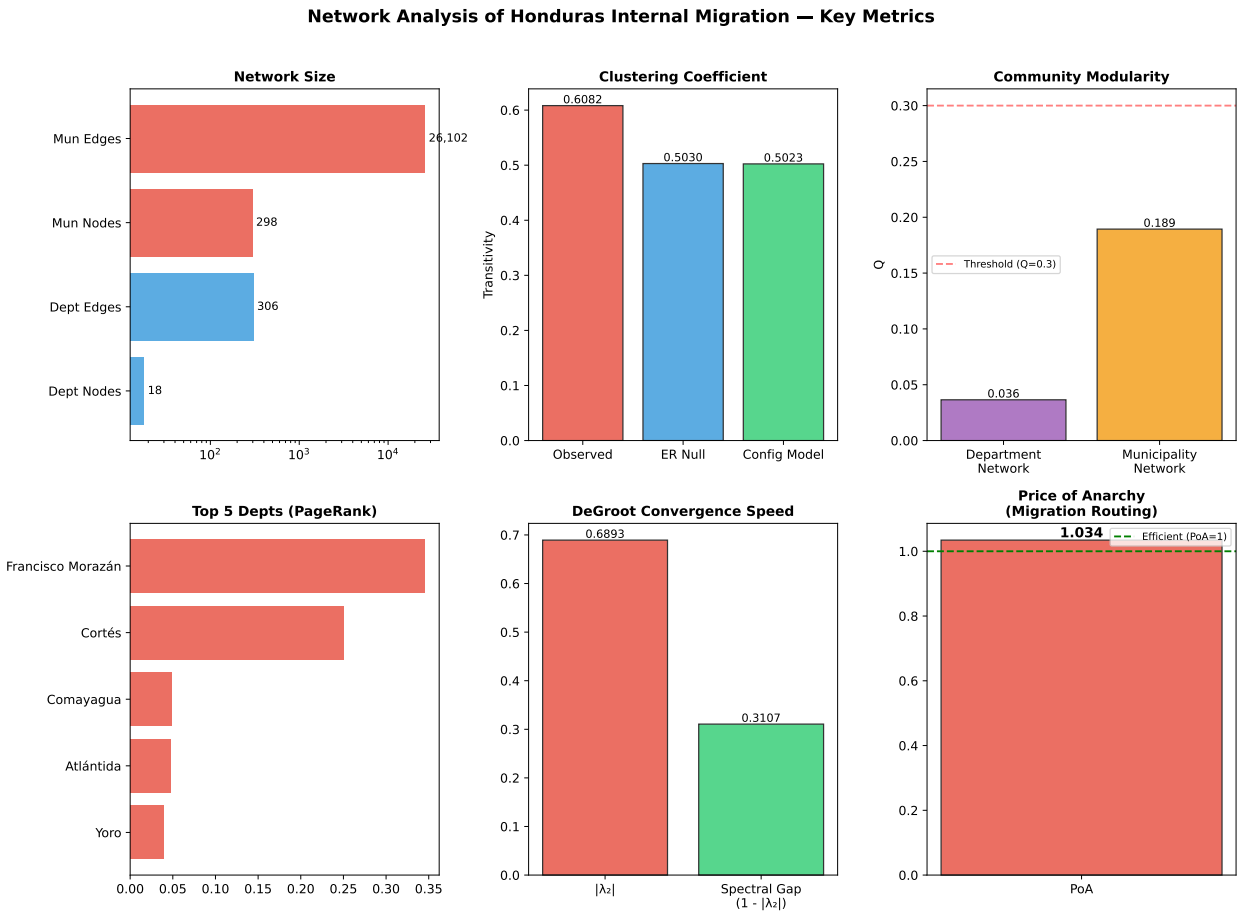


Figure 23: Summary dashboard: key network metrics at a glance

```
#   Summary table
summary = {
    'Analysis': [
        'Graph Construction',
        'Degree Distribution',
        'Centrality (PageRank)',
        'Null Model Comparison',
        'Community Detection',
        'SI Spreading',
        'Threshold Model',
        'Optimal Seeding',
        'DeGroot Dynamics',
        'Friedkin-Johnsen',
        'Routing Analogy'
    ],
    'Course Unit': [
        'Unit 1', 'Unit 1-2', 'Unit 1', 'Unit 2',
        'Unit 3', 'Unit 5', 'Unit 5.2', 'Unit 5.2',
        'Unit 6', 'Unit 6', 'Unit 4'
    ],
    'Key Finding': [
        f'{G_dept.number_of_nodes()} dept nodes, {G_dept.number_of_edges()} edges; '
```

```
        f'{G_mun_connected.number_of_nodes()} municipality nodes',
        'Heavy-tailed: few municipalities attract most migrants',
        'Tegucigalpa & SPS dominate; betweenness reveals bridge departments',
        f'Clustering {obs_clustering:.3f} >> ER random ({np.mean(er_clustering):.3f})',
        f'Modularity Q = {Q:.3f}; regional migration clusters identified',
        'Migration contagion spreads faster from high-centrality seeds',
        'Complex contagion: reinforcement needed for migration cascades',
        'Greedy algorithm identifies optimal intervention targets',
        f'Convergence to consensus; | | = {lam2:.3f}',
        'Persistent disagreement: isolated departments keep own rates',
        f'Price of Anarchy = {poa:.3f}; congestion externalities' if T_total > 0 else 'Congestion extern
    ]
}

summary_df = pd.DataFrame(summary)
print(summary_df.to_string(index=False))
```

```
             Analysis Course Unit                                                    Key Finding
   Graph Construction      Unit 1                    18 dept nodes, 306 edges; 298 municipality nodes
  Degree Distribution    Unit 1-2             Heavy-tailed: few municipalities attract most migrants
Centrality (PageRank)      Unit 1 Tegucigalpa & SPS dominate; betweenness reveals bridge departments
Null Model Comparison      Unit 2                                Clustering 0.608 >> ER random (0.503)
  Community Detection      Unit 3       Modularity Q = 0.036; regional migration clusters identified
         SI Spreading      Unit 5      Migration contagion spreads faster from high-
centrality seeds
      Threshold Model    Unit 5.2     Complex contagion: reinforcement needed for migration cascades
      Optimal Seeding    Unit 5.2           Greedy algorithm identifies optimal intervention targets
     DeGroot Dynamics      Unit 6                              Convergence to consensus; | | = 0.689
     Friedkin-Johnsen      Unit 6       Persistent disagreement: isolated departments keep own rates
      Routing Analogy      Unit 4                 Price of Anarchy = 1.034; congestion externalities
```

---

# 13 Conclusion

This analysis demonstrates that the Honduras remittance dataset is a rich source for **network analysis**, naturally mapping onto a directed weighted graph of internal migration. Applying the full toolkit from the Networks I course reveals:

1. **Structural properties** (Units 1–2): The migration network is far from random — it exhibits high clustering, heavy-tailed degree distributions, and small-world properties, all consistent with geographic gravity and preferential attachment. The origin–destination heatmap and net migration balance bar chart reveal a strongly **asymmetric, hub-dominated** system where a few departments receive the majority of internal migrants.

2. **Centrality** (Unit 1): PageRank and betweenness centrality identify distinct department roles: terminal attractors (large cities), transit hubs (bridge departments that connect migration clusters), and peripheral senders. These roles map directly onto different policy needs — absorptive capacity investments for attractors, transit services for bridges, and economic development for senders.

3. **Community structure** (Unit 3): Migration flows are regionally clustered, with departments forming distinct migration communities centred on the two main urban poles. The community composition

heatmap reveals whether these clusters align with administrative boundaries or cross them, informing the design of regional development zones.

4. **Diffusion dynamics** (Units 5–5.2): Migration can be modelled as a spreading process. The SI model identifies epidemic thresholds and super-spreader departments. The threshold model reveals that complex contagion (social reinforcement) better explains migration decisions than simple contagion, and greedy influence maximisation identifies optimal municipalities for targeted development interventions using the Kempe-Kleinberg-Tardos framework.

5. **Opinion dynamics** (Unit 6): The DeGroot and Friedkin-Johnsen models show how migration propensities converge (or persist) through network interactions. The listening matrix $T$ and its spectral properties determine convergence speed and influence distribution. The FJ model reveals persistent disagreement: isolated departments maintain their own migration dynamics regardless of national trends, while connected departments are susceptible to network-mediated social influence.

6. **Routing equilibrium** (Unit 4): Migration flows create congestion externalities in destination cities, analogous to selfish routing. The Price of Anarchy quantifies the inefficiency of decentralised migration choices and provides a dollar-value argument for active regional development policy.

## 13.1 Development Policy Recommendations

**!** Integrated Policy Framework Based on Network Analysis

The network analysis of Honduras's internal migration yields a coherent set of **evidence-based policy recommendations**:

**1. Invest in secondary cities (decentralisation).** The hub-and-spoke structure, heavy-tailed degree distribution, and routing congestion analysis all point to the same conclusion: Honduras's migration system is dangerously over-concentrated in two cities. Investing in Comayagua, La Ceiba, Choluteca, and other intermediate urban centres would create alternative migration destinations, reduce congestion costs, and improve national resilience.

**2. Design regional development zones around migration communities, not administrative boundaries.** The Louvain community detection reveals that economic regions (as revealed by actual population movements) may differ from departmental boundaries. Development coordination mechanisms should follow these data-driven clusters.

**3. Use the greedy influence maximisation algorithm to prioritise intervention locations.** With limited budgets, the optimal seeding approach identifies which specific departments provide the maximum cascade effect for development interventions. This is a departure from population-based or poverty-based targeting — **network position determines impact**.

**4. Tailor policies to department susceptibility (Friedkin-Johnsen $\alpha_i$).** Well-connected departments respond to network-level interventions (information campaigns, peer effects); isolated departments require direct, local investments. One-size-fits-all national programs will underperform.

**5. Address congestion externalities explicitly.** The routing analysis shows that individual migration decisions impose costs on others. Policies that internalise these externalities — subsidised relocation to secondary cities, congestion-based urban service pricing, or proactive capacity expansion in growing areas — can close the gap between Nash equilibrium and social optimum.

**6. Monitor the spectral gap for early warning.** A shrinking spectral gap of the migration network's listening matrix indicates increasing fragmentation of the migration system. Tracking this metric over time provides an early warning signal for policymakers.

# 14 References


- Kempe, D., Kleinberg, J., & Tardos, É. (2003). Maximizing the spread of influence through a social network. *KDD*.
- Beaman, L., BenYishay, A., Magruder, J., & Mobarak, A. M. (2018). Can network theory-based targeting increase technology adoption? NBER WP 24912.
- DeGroot, M. (1974). Reaching a consensus. *JASA*.
- Friedkin, N. & Johnsen, E. (1990). Social influence and opinions. *J. Math. Sociology*.
- Golub, B. & Jackson, M. (2010). Naïve learning in social networks and the wisdom of crowds. *AEJ: Micro*.
- Youn, H., Gastner, M. & Jeong, H. (2008). Price of anarchy in transportation networks. *PRL*.
- Bryan, G., Chowdhury, S. & Mobarak, A. M. (2014). Underinvestment in a profitable technology: The case of seasonal migration in Bangladesh. *Econometrica*.
- Milán, P. (2026). Networks I: Concepts and Algorithms. BSE Lecture Notes.